

TECHNICAL DESIGN DOCUMENT

PulsarNav AI Technical Design Document

Autonomous Pulsar-Based Deep Space Navigation Framework

Project	PulsarNav AI
Repository	/Users/supryo/Desktop/pulsar
Prepared	2026-07-01
Document Class	Technical Design Document
Primary Audience	Academic reviewers, aerospace mentors, flight dynamics developers

This document describes PulsarNav AI as a scientific software system: why each component exists, the mathematics behind it, how it is implemented, and how its outcomes are validated.

Document Map

1. Executive documentation and project outcomes
2. Mathematical foundations
3. Software architecture and module descriptions
4. Data pipeline and validation approach
5. User interface and visualization documentation
6. Future work, references, and appendices

PulsarNav AI: Autonomous Pulsar-Based Deep Space Navigation Framework

Complete Engineering & Scientific Handbook

This document serves as the comprehensive technical guide and architectural handbook for PulsarNav AI, an autonomous spacecraft navigation framework utilizing high-stability millisecond pulsars (MSPs) and the NANOGrav 12.5-Year Narrowband Dataset. It is designed for SAC ISRO mentors, future developers, academic researchers, and students joining the project.

1. Project Overview

Project Objective

The primary objective of PulsarNav AI is to design, implement, and validate a research-grade simulation and estimation framework for X-ray Pulsar Navigation (XNAV). The project integrates a high-fidelity Python scientific pipeline for astronomical data parsing, signal processing, and orbit determination with a modern, interactive Next.js mission-control console that provides real-time 3D orbit visualization, parameter sensitivity analysis, and statistical error validation.

Problem Statement

Deep-space exploration missions (e.g., lunar, Martian, and outer-planet probes) have traditionally relied on Earth-based tracking systems. The ground-based paradigm introduces several severe operational bottlenecks:

7. Geometric Dilution & Error Scaling: Ground-based angular measurements (using VLBI) and range measurements suffer from errors that scale linearly with the spacecraft's distance from Earth. At outer-planet distances (e.g., Jupiter or Saturn), position errors can expand to tens or hundreds of kilometers.
8. Communication Latency: Two-way light time delays between Earth and deep space prevent real-time autonomous orbital corrections, which are critical during entry, descent, and landing (EDL) or high-velocity flybys.
9. Network Congestion: The Deep Space Network (DSN) is an expensive, oversubscribed global resource. Scheduling antenna time for routine orbit determination limits tracking frequency and creates operational bottlenecks.

Importance of Pulsar Navigation

Pulsar navigation (XNAV) solves these challenges by utilizing celestial beacon sources—specifically, millisecond pulsars (MSPs). MSPs are rapidly rotating neutron stars with rotation periods of 1 to 10 milliseconds. They emit highly directional beams of electromagnetic radiation from their magnetic poles. Due to their immense density and conservation of angular momentum, their spin stability is comparable to terrestrial atomic clocks (10^{-19} s/s spin-down rates).

By measuring the times of arrival (TOAs) of pulsar pulses at a spacecraft, the vehicle can determine its position, velocity, and onboard clock bias completely autonomously. Crucially, because pulsars are distributed isotropically across the galaxy, the navigation error does not scale with distance from Earth. XNAV provides a uniform, autonomous 3D positioning grid across the entire solar system.

Comparative Analysis: GPS vs. DSN vs. Pulsar Navigation (XNAV)

Parameter	Global Positioning System (GPS)	Deep Space Network (DSN)	Pulsar Navigation (XNAV)
Signal Source	Earth-orbiting artificial satellites	Earth-based giant antenna arrays	Galactic millisecond pulsars (neutron stars)
Operational Region	Near-Earth space (LEO, MEO, up to GEO)	Solar system and interstellar space	Unlimited (anywhere in the heliosphere and beyond)
Autonomy	Fully autonomous at receiver level	Dependent on ground-station uplinks/downlinks	100% autonomous onboard spacecraft
Error Characteristics	~ 1 - 10 meters, degrades above GEO	Scales linearly with distance from Earth	Uniform (~ 10 m to 10 km depending on integration time)
Frequencies	L-band radio (microwave)	S, X, and Ka-band radio	X-ray (highly collimated) or Radio
Availability	Continuous near Earth	Scheduled bottlenecks, subject to Earth rotation	Continuous, omnidirectional

2. Repository Architecture

The PulsarNav AI codebase is bifurcated into a high-fidelity Python scientific library (`pulsar_nav`) and a Next.js web application for visual simulation and interactive telemetry analysis.

```
.
├── app/                # Next.js App Router Pages & API Routes
│   ├── api/
│   │   └── navigation-lab/
│   │       └── route.ts    # Web-based Monte Carlo & EKF Solver API
│   ├── dashboard/
│   │   └── page.tsx        # Mission Control Dashboard Shell
│   ├── globals.css        # Tailwind styles with RGB theme variables
│   ├── layout.tsx         # Main React Context providers
│   └── page.tsx           # Landing Page & Initial Parameter Config
├── components/           # Reusable React & Three.js UI Components
│   ├── charts.tsx         # Recharts wrappers for error summaries & CDFs
│   ├── navigation-comparison.tsx  # Interactive dashboard controls & comparisons
│   └── space-scene.tsx     # Three.js / React Three Fiber J2000 3D canvas
```

└──	star-field.tsx	# Starry background particle effect
└──	ui.tsx	# Shorthand styled DOM elements
──	lib/	# Core UI state hooks & local assets
──	pulsar_nav/	# Core Scientific Python Library
└──	init.py	# Package entry point
└──	cli.py	# Python pipeline CLI launcher
└──	config.py	# Physical & numerical constants
└──	coordinates.py	# RA/Dec to Cartesian coordinate converters
└──	crlb.py	# Fisher Information & Cramér-Rao Lower Bounds
└──	delay_estimation.py	# CC, NLS, and MLE phase delay algorithms
└──	epoch_folding.py	# Epoch-folding & velocity-error tolerances
└──	kalman_filter.py	# 10-state linear error-state Kalman Filter
└──	navigation_engine.py	# WLS / LS position estimation pipeline
└──	navigation_geometry.py	# Direction matrix Gamma & observability
└──	parsing.py	# TEMPO/TEMPO2 .par & .tim parser
└──	pulsar_catalog.py	# Astro reference catalog for the 8 book sources
└──	pulse_delay_estimation.py	# Test compatibility adapter layer
└──	ranking.py	# Pulsar multi-factor ranking & GDOP selector
└──	signal_model.py	# NHPP rate functions & photon TOA simulator
└──	simulation.py	# Trajectory simulation helper functions
└──	timing_model.py	# Barycentric Römer & dispersion delays
└──	xray_nav.py	# High-fidelity 8-state absolute EKF engine
──	sample_data/	# Packaged lightweight CSV files for Git
──	scripts/	# Command-line utility scripts
└──	run_navigation_lab.py	# Python Monte Carlo script
└──	run_pipeline.py	# Python batch pipeline runner
──	tests/	# Python automated test suite
└──	test_navigation_engine.py	# Verification for LS/WLS engines
└──	test_pulse_delay_estimation.py	# Verification for phase estimators
└──	test_timing.py	# Verification for Römer & dispersion models
└──	test_xray_nav.py	# Verification for orbital EKF dynamics

└── package.json	# Node.json dependency manifest
└── requirements.txt	# Python dependency manifest

Module Descriptions

- `coordinates.py`: Parses raw astronomical coordinates (e.g., sexagesimal right ascension and declination strings) and maps them to 3D Cartesian unit vectors in the Equatorial J2000 reference frame.
- `timing_model.py`: Calculates barycentric time corrections. Implements the Keplerian analytical orbit of the Earth-Moon Barycenter (EMB) relative to the Solar System Barycenter (SSB), Römer delays, and interstellar dispersion delays ($1/f^2$ plasma dispersion).
- `signal_model.py`: Simulates individual photon arrivals at a detector. Implements a Non-Homogeneous Poisson Process (NHPP) using accumulated rate inversion (Newton-Raphson method).
- `epoch_folding.py`: Implements phase binning of photon TOAs. Computes the empirical rate function and evaluates the smearing impact of velocity errors.
- `delay_estimation.py`: Implements three distinct phase delay estimation techniques: Cross-Correlation (FFT-accelerated), Nonlinear Least Squares (NLS), and Maximum Likelihood Estimation (MLE).
- `crlb.py`: Calculates the theoretical limits of timing accuracy by evaluating the Fisher Information integral. Computes Asymptotic Relative Efficiency (ARE) to measure information loss due to phase binning.
- `navigation_geometry.py`: Evaluates system observability (system matrix F and measurement matrix H). Computes the Geometric Dilution of Precision (GDOP) and Position Dilution of Precision (PDOP).
- `kalman_filter.py`: Formulates a 10-state linear error-state Kalman Filter (L-ESKF) for tracking spacecraft position, velocity, accelerometer biases, and clock bias relative to a nominal IMU trajectory.
- `xray_nav.py`: Implements a high-fidelity, absolute-state 8-state Extended Kalman Filter (EKF) featuring full orbital gravity models (two-body Keplerian + Earth J_2 oblateness) propagated via a 4th-order Runge-Kutta (RK4) integrator.

3. Dataset Overview

NANOGrav 12.5-Year Narrowband Dataset

High-precision pulsar navigation models must be grounded in real astronomical measurements. PulsarNav AI is designed to ingest the official NANOGrav 12.5-Year Data Release, which contains high-cadence, sub-microsecond timing observations of dozens of millisecond pulsars collected using the Arecibo Observatory and the Green Bank Telescope.

Telescope Parameter (`.par`) Files

A `.par` file contains the physical spin and astrometric model of the pulsar, derived from decades of phase-coherent timing. Key parameters extracted by our parser include:

- ``PSRJ` / `PSR``: The official pulsar name (based on J2000 or B1950 coordinates).
- ``RAJ` / `DECJ``: Right Ascension and Declination (sexagesimal strings, e.g., ``17:13:49.533` / `+07:47:37.49``).
- ``F0``: The spin frequency (f_0 in Hz).
- ``F1``: The first derivative of the spin frequency ($\dot{f}$ in Hz/s, tracking energy loss due to magnetic dipole radiation).
- ``PEPOCH``: The reference epoch at which the spin frequency is defined (MJD).
- ``DM``: Dispersion Measure (pc/cm³, representing the integrated electron column density along the line of sight).
- ``START` / `FINISH``: The Modified Julian Dates defining the observation span.

Example .par File Structure (Abridged)

```

PSRJ      J1713+0747
RAJ       17:13:49.533470
DECJ      07:47:37.49512
F0        218.811843829054 1
F1        -4.08375e-16    1
PEPOCH    54500.000000
DM        15.9875        1

```

Time of Arrival (`.tim`) Files

A `.tim` file contains lists of individual pulse Times of Arrival (TOAs) observed at the telescope. Each row represents a single TOA, formatted under TEMPO2 standards (typically `FORMAT 1` or `PRINCETON` style):

- The raw observed frequency (MHz) of the incoming radio wave.
- The Modified Julian Date (MJD) of the TOA at the observatory.
- The measurement uncertainty (microseconds) of the TOA.
- The observatory code (e.g., ``ao`` for Arecibo, ``gbt`` for Green Bank).
- Flags detailing instrument type, receiver backend, and channel bandwidth.

Example .tim File Structure (Abridged)

```

FORMAT 1

gbt_file.raw 1400.000000 55120.354182910543 0.035 gbt -fe L-wide -be PUPPI
ao_file.raw  430.000000 55121.109284102941 0.082 ao  -fe 430  -be ASP

```

Data Flow Through the Ingestion System

The ingestion and parsing pipeline flows as follows:

```

[ .par Files ] -----> [ parse_par_file ] -----> [ Catalog DataFrame ] -----> [ Unit Vectors ]
|
v

[ .tim Files ] -----> [ parse_tim_file ] -----> [ TOA DataFrame ] -----> [ Ranking Engine ]
|               |
v               v

[ TOA Statistics ] -----> [ Selected Pulsars ]

```

10. Catalog Construction: ``build_catalog`` in `parsing.py` reads the ``par`` directory, extracts the astrometric and spin coordinates, and builds a unified pulsar catalog.
11. Direction Vector Computation: ``build_unit_vectors`` in `parsing.py` converts the catalog's celestial angles (RA/Dec) into Equatorial J2000 unit vectors `n`.

12. TOA Aggregation: `build_toa_database` in `parsing.py` parses the `.tim` files, compiling millions of TOA rows into a single database.
13. Statistical Summarization: `toa_statistics` in `parsing.py` groups the database by pulsar name, calculating the total number of TOAs, observation duration (days), median frequency, and median measurement error.

4. Pulsar Concepts

Physics of Pulsars

Pulsars are highly magnetized, rotating neutron stars formed during the core-collapse supernova explosions of massive stars (typically 8 to 25 solar masses). A neutron star packages approximately 1.4 to 2.1 solar masses into a sphere of only 10 to 12 kilometers radius. The star features an intense magnetic field (10^8 to 10^{12} Gauss).

Relativistic particles are accelerated along the magnetic dipole axis, producing beams of synchrotron and curvature radiation. If the magnetic axis is misaligned with the rotational axis, the beam sweeps across space like a lighthouse. When the beam crosses Earth's line of sight, telescopes register a periodic pulse.

Millisecond Pulsars (MSPs)

MSPs are a sub-class of pulsars that have been "spun up" to sub-second periods through the accretion of matter and angular momentum from a binary companion star (a process known as recycling). MSP periods range from 1.4 ms (e.g., PSR B1937+21 spinning at ~ 642 Hz) up to ~ 10 ms.

Because their magnetic field strength is lower than young pulsars (10^8 Gauss vs. 10^{12} Gauss), they experience minimal torque and spin down at extremely slow rates ($\dot{\omega} \sim 10^{-19}$ to 10^{-21} Hz/s). This long-term rotational stability makes them ideal deep-space clock beacons.

Pulsar Timing

Pulsar timing models the rotation of the pulsar and accounts for the propagation of pulses to the solar system. The rotational phase $\phi(t)$ of the pulsar at its emission time is represented as a Taylor series:

$$\phi(t) = \phi(t_0) + f_0 \cdot (t - t_0) + \frac{1}{2} \dot{f}_0 \cdot (t - t_0)^2 + \frac{1}{6} \ddot{f}_0 \cdot (t - t_0)^3 + \dots$$

Because the spin-down terms ($\dot{f}_0$, $\ddot{f}_0$) are extremely small for MSPs, the linear term dominates over timescales of years, simplifying navigation calculations.

Pulse Arrival Times & Timing Residuals

When pulses are observed, the recorded Time of Arrival (TOA) at the detector (t_{obs}) is compared to the expected TOA computed from the timing model (t_{pred}). The difference forms the timing residual $\mathcal{R}(t)$:

$$\mathcal{R}(t) = t_{\text{obs}} - t_{\text{pred}}$$

- If the timing model is perfect, the residuals should consist only of zero-mean white noise (representing detector measurement error).
- Systematic deviations in residuals indicate physical effects:
 - A linear slope indicates a clock drift or period error.
 - A sinusoidal curve with a period of one Earth year indicates an error in the assumed position of the observer (e.g., Earth coordinate error or spacecraft trajectory perturbation).
- In XNAV, these residuals are the primary observables used to correct the spacecraft state.

5. Coordinate Systems

Coordinate systems are essential to translate pulsar positions into directions and to resolve spacecraft trajectories.

Celestial North (+Z)

^

| . Pulsar (n)

| /|

| / | Declination (dec)

| / |

|/_> Equatorial Plane (X-Y)

/ \

/ \ Right Ascension (ra)

/ \

Vernal v

Equinox (+X) (+Y)

Right Ascension (RA) and Declination (DEC)

The standard astronomical coordinate system is the Geocentric Equatorial Frame (J2000.0).

- Vernal Equinox (+X): The direction from the Earth center to the Sun center at the moment of the vernal equinox.
- Celestial North Pole (+Z): Parallel to Earth's rotation axis.
- Right Ascension (α or RA): The angle measured eastward along the celestial equator from the vernal equinox. Expressed in hours, minutes, and seconds (24 hours = 360°), or in degrees (1 hour = 15°).
- Declination (δ or DEC): The angle measured north (+) or south (-) of the celestial equator, ranging from -90° to $+90^\circ$.

Ecliptic Coordinate System

In planetary navigation, the Ecliptic Coordinate System is centered on the Sun or Earth, but uses Earth's orbital plane as the reference equator ($Z=0$).

- Ecliptic Longitude (λ): Measured along the ecliptic plane eastward from the vernal equinox.
- Ecliptic Latitude (β): Measured perpendicular to the ecliptic plane.

The tilt between Earth's equator and the ecliptic plane is the obliquity of the ecliptic ($\epsilon \sim 23.43929111^\circ$). The transformation from ecliptic coordinates (λ, β) to equatorial coordinates (α, δ) is implemented in `coordinates.py` via:

$$\sin(\delta) = \sin(\beta)\cos(\epsilon) + \cos(\beta)\sin(\epsilon)\sin(\lambda)$$

$$\tan(\alpha) = \frac{\sin(\lambda)\cos(\epsilon) - \tan(\beta)\sin(\epsilon)\cos(\lambda)}{\cos(\lambda)\cos(\epsilon) + \tan(\beta)\sin(\epsilon)\sin(\lambda)}$$

Conversion to Cartesian Unit Vectors

To compute Römer delays, we must convert RA (α) and Dec (δ) to a Cartesian unit vector $\mathbf{n}$ pointing from the origin toward the pulsar:

$$\mathbf{n} = \begin{bmatrix} n_x \\ n_y \\ n_z \end{bmatrix} = \begin{bmatrix} \cos(\delta)\cos(\alpha) \\ \cos(\delta)\sin(\alpha) \\ \sin(\delta) \end{bmatrix}$$

This conversion is implemented in `sky_to_unit_vector` in `coordinates.py`.

6. Navigation Geometry

Direction and Spacecraft Position Vectors

Let $\mathbf{r}$ be the spacecraft position vector relative to the coordinate origin. Let $\mathbf{n}_i$ be the unit direction vector pointing toward pulsar i . Because pulsars are at interstellar distances (typically hundreds to thousands of light-years), the direction vector $\mathbf{n}_i$ is parallel at all points within the solar system.

Timing Delay Equations (Römer Delay)

A wavefront emitted by pulsar i passes the Solar System Barycenter (SSB) at time t_{ssb} . The arrival time of the same wavefront at the spacecraft (t_{sc}) is shifted due to the light-travel time across the distance separating the spacecraft and the SSB:

$$t_{\text{sc}} - t_{\text{ssb}} = \frac{r}{c} \cdot \cos(\theta) = \frac{\mathbf{r} \cdot \mathbf{n}_i}{c}$$

where c is the speed of light. This geometric offset is the Römer delay $\Delta t_{R,i}$.

Incoming Plane Wavefronts

| | | | |

$\mathbf{v} \quad \mathbf{v} \quad \mathbf{v} \quad \mathbf{v} \quad \mathbf{v}$ Direction vector $\mathbf{n}$

===== /

/ /

/ /

/ /

/ /

/ $\mathbf{v}$

* Spacecraft ($\mathbf{r}$)

/

/

/ $\mathbf{r} \cdot \mathbf{n}$ (Projection distance)

/

$\mathbf{v}$

* Barycenter (SSB)

If we observe from a spacecraft with position $\mathbf{r}$ and an onboard clock that suffers from a bias error b (seconds), the measured arrival time delay z_i relative to the predicted arrival time at the origin is:

$$z_i = \frac{\mathbf{r} \cdot \mathbf{n}_i}{c} + b + \eta_i$$

where η_i represents timing noise. Multiplying by the speed of light c :

$$z_i \cdot c = \mathbf{r} \cdot \mathbf{n}_i + c \cdot b + c \cdot \eta_i$$

Geometric Interpretation

Each pulsar measurement provides the projection of the spacecraft's 3D position onto a 1D line of sight: $\mathbf{x}_i = \mathbf{r} \cdot \mathbf{n}_i$.

- A single pulsar measurement constrains the spacecraft to a plane perpendicular to $\mathbf{n}_i$.
- Two non-parallel pulsars constrain the spacecraft to the intersection line of two planes.
- Three non-coplanar pulsars constrain the spacecraft to a single point in 3D space, assuming the clock is perfectly synchronized ($b=0$).
- Since the onboard atomic clock has an unknown bias b , we have 4 unknowns (r_x, r_y, r_z , and $c \cdot b$). Therefore, we require at least 4 pulsars with linearly independent direction vectors to solve for both 3D position and clock bias simultaneously.

7. Signal Processing Pipeline

The signal processing pipeline models the transition from physical photon arrivals to timing estimates.

Signal Modeling

The rate of photon arrivals at an onboard detector is modeled as a time-varying Poisson process. According to Emadzadeh & Speyer (2011), the intensity rate function $\lambda(t)$ is:

$$\lambda(t) = \lambda_b + \lambda_s \cdot h(\phi(t))$$

where:

- λ_b is the background photon flux (cosmic X-ray background, detector dark currents).
- λ_s is the source pulse flux.
- $h(\phi)$ is the normalized pulse profile, satisfying $\int_0^1 h(\phi) d\phi = 1$.
- $\phi(t)$ is the pulse phase at the detector, representing the fractional cycle of the pulsar.

Non-Homogeneous Poisson Process (NHPP)

Because X-ray photon arrivals are discrete events, the probability of observing k photons within an interval $[t_a, t_b]$ is given by:

$$P(k \text{ photons in } [t_a, t_b]) = \frac{[\Lambda(t_a, t_b)]^k e^{-\Lambda(t_a, t_b)}}{k!}$$

where $\Lambda(t_a, t_b)$ is the integrated rate function:

$$\Lambda(t_a, t_b) = \int_{t_a}^{t_b} \lambda(\tau) d\tau = \lambda_b (t_b - t_a) + \int_{t_a}^{t_b} \lambda_s h(\phi(\tau)) d\tau$$

Photon Arrival Simulation (Algorithm 3.1)

Simulating photon arrival times is critical for testing phase estimators. We generate non-homogeneous arrival times by inverting the integrated rate function. This is implemented in `generate_photon_toas` in `signal_model.py`:

$$\Lambda(t_{n+1}) = \Lambda(t_n) + E_n, \text{ where } E_n \sim \text{Exponential}(1)$$

To find t_{n+1} , we solve $\Lambda(t_{n+1}) - y_{\text{target}} = 0$ using Newton-Raphson iteration:

$$t_{n+1}^{(i+1)} = t_{n+1}^{(i)} - \frac{\Lambda(t_{n+1}^{(i)}) - y_{\text{target}}}{\lambda(t_{n+1}^{(i)})}$$

8. Epoch Folding

Purpose

X-ray detectors typically observe very weak signals. For instance, the Crab pulsar might produce only $\lambda_s = 15$ ph/s on a 1 m^2 detector against a background of $\lambda_b = 5$ ph/s. With a period of 33 ms, we receive an average of only 0.66 photons per period. It is impossible to reconstruct the pulse shape or estimate its arrival time from a single pulse.

Epoch Folding circumvents this by cutting the continuous photon arrival timeline into segments of length $P = 1/f_{\text{obs}}$ and stacking them. This accumulates signal coherently while noise averages out.

Continuous TOAs: ----*---*-----*---*---*-----*---*--

Fold modulo P: | * | * | * | * | * | * |

Stacked Profile: [| * | | * | * | *] --> Histogram into Bins

Mathematical Intuition

For each photon TOA t_j , we compute its fractional phase $\phi_j \in [0, 1)$:

$$\phi_j = (t_j - t_0) \cdot f_{\text{obs}} \bmod 1.0$$

We partition the phase interval $[0, 1)$ into N_b bins of width $\Delta\phi = 1/N_b$. The photons are histogrammed into these bins. The empirical rate function $\bar{\lambda}(t_i)$ in bin i is normalized as (Eq 3.39):

$$\bar{\lambda}(t_i) = \frac{C_i}{N_b T_{\text{obs}}}$$

where C_i is the photon count in bin i , and T_{obs} is the total observation duration.

According to Theorem 3.2, the variance of this empirical rate estimator is (Eq 3.42):

$$\text{Var}[\bar{\lambda}(t_i)] = \frac{N_b T_{\text{obs}}}{C_i} \lambda(t_i)$$

This shows that the noise variance decreases as $1/T_{\text{obs}}$, improving the signal-to-noise ratio.

Velocity Error Smearing

If the spacecraft's velocity relative to the pulsar is wrong by Δv , the assumed folding frequency is wrong by $\Delta f_{\text{obs}} = f_s \frac{\Delta v}{c}$. This causes a progressive phase drift per period:

$$\Delta\phi = -\frac{\Delta v}{c}$$

Across N_p periods, the cumulative phase drift is $N_p \Delta \phi$. If this drift exceeds half a bin width ($1/(2N_b)$), the peak of the folded profile becomes smeared, degrading timing accuracy. The velocity error tolerance to prevent smearing is (Eq 3.72):

$$|\Delta v| < \frac{1}{2N_b} \cdot N_p$$

9. Delay Estimation

After folding, we estimate the initial phase shift ϕ_0 that aligns the observed profile with the template. The delay is then computed as $t_d = \phi_0 / f_s$.

Cross-Correlation (CC)

The Cross-Correlation estimator correlates the empirical rate $\bar{\lambda}$ with the true template $h(\phi)$ to find the phase shift that maximizes the correlation. The CC cost function is:

$$R_D(\psi) = \frac{1}{N_b} \sum_{k=0}^{N_b-1} (\bar{\lambda}(t_k) - \langle \bar{\lambda} \rangle) (h(\phi_k + \psi) - \langle h \rangle)$$

We compute this correlation using the Fast Fourier Transform (FFT) in $O(N_b \log N_b)$ time:

$$R_D = \text{Real}(\text{IFFT}(\text{FFT}(\bar{\lambda}) \cdot \text{conj}(\text{FFT}(h)))$$

The peak index k_m is refined to sub-bin resolution using parabolic interpolation:

$$\delta = -0.5 \frac{R_D(k_{m+1}) - R_D(k_m-1)R_D(k_{m+1})}{2R_D(k_m) + R_D(k_{m-1})}$$

The final phase estimate is:

$$\phi_0 = -\frac{k_m}{N_b} + \delta \cdot N_b \bmod 1.0$$

Nonlinear Least Squares (NLS)

The NLS estimator finds the phase shift ϕ that minimizes the squared residuals between the empirical profile and the scaled template:

$$J(\phi) = \sum_{i=1}^{N_b} (\bar{\lambda}(t_i) - (\lambda_b + \lambda_s h(\text{bin}_i + \phi)))^2$$

In `delay_estimation.py`, this is solved by performing a grid search over $[0, 1)$ to find the global minimum, followed by local refinement using Brent's method.

Maximum Likelihood Estimation (MLE)

The MLE operates directly on raw photon TOAs ($t_1, t_2, \dots, t_{N_p}$) without binning or folding. The log-likelihood function $\Psi(\phi)$ is (Eq 6.5):

$$\Psi(\phi) = \sum_{j=1}^{N_{\text{photons}}} \ln(\lambda_b + \lambda_s h(f_{\text{obs}} t_j + \phi)) - \lambda(T_{\text{obs}})$$

Since $\lambda(T_{\text{obs}}) \sim T_{\text{obs}}(\lambda_b + \lambda_s)$ is nearly independent of ϕ for long observations, we maximize the sum. This estimator is statistically optimal and asymptotically efficient (achieving the CRLB), but it is more computationally intensive.

10. Navigation Engine

Least Squares Formulation

For $N \geq 4$ pulsars, the relationship between spacecraft position r , clock bias b , and measured delays z_i can be written as:

$$A w = b$$

$$\begin{bmatrix} n_{1,x} & n_{1,y} & n_{1,z} & 1 \\ n_{2,x} & n_{2,y} & n_{2,z} & 1 \\ \dots & \dots & \dots & \dots \\ n_{N,x} & n_{N,y} & n_{N,z} & 1 \end{bmatrix} \begin{bmatrix} r_x \\ r_y \\ r_z \\ c \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \dots \\ z_N \end{bmatrix}$$

This is solved via the pseudoinverse:

$$w = (A^T A)^{-1} A^T b$$

Weighted Least Squares (WLS)

Since different pulsars have different timing accuracies (e.g., Crab is bright but has high timing noise, while MSPs are quiet but stable), we weight the measurements by their inverse variances:

$$W = \text{diag}(\frac{1}{\sigma_1^2}, \frac{1}{\sigma_2^2}, \dots, \frac{1}{\sigma_N^2})$$

where $\sigma_i = c \cdot \sqrt{\text{CRLB}(t_{d,i})}$ is the position uncertainty (meters) along the line of sight to pulsar i . The weighted least squares solution is:

$$w_{\text{wls}} = (A^T W A)^{-1} A^T W b$$

11. Kalman Filtering

PulsarNav AI contains two Kalman Filter implementations: a 10-state linear error-state filter and an 8-state absolute Extended Kalman Filter.

10-State Linear Error-State Kalman Filter (L-ESKF)

Implemented in `kalman_filter.py`, this filter tracks the deviations between the true spacecraft trajectory and a nominal reference trajectory (e.g., dead-reckoning from an Inertial Measurement Unit):

$$X = \begin{bmatrix} \Delta x & \Delta v_x & \Delta v_y & \Delta v_z & b_a & t_e \end{bmatrix}^T$$

where:

- $\Delta x, \Delta v$ are position and velocity errors.
- b_a is the 3D accelerometer bias.
- t_e is the clock bias.

Continuous-Time System Dynamics

$$\dot{X}(t) = F X(t) + w(t)$$

$$F = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Because $F^3 = 0$ (nilpotent), the discrete state transition matrix $\Phi = e^{F T_s}$ simplifies to:

$$\Phi = \begin{bmatrix} I & T_s I & \frac{1}{2} T_s^2 I & 0 \\ 0 & I & T_s I & 0 \\ 0 & 0 & I & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Process Noise Covariance Q

The discrete-time process noise covariance Q is obtained by integrating the continuous-time noise spectral density matrix $Q_c = \text{diag}(W_v, W_a, W_b, W_e)$:

$$Q = \int_0^{T_s} \Phi(\tau) Q_c \Phi^T(\tau) d\tau$$

The blocks along the diagonal grow with powers of the time step T_s :

- Position error variance: $T_s W_v + \frac{1}{3} T_s^3 W_a + \frac{1}{20} T_s^5 W_b$.
- Velocity error variance: $T_s W_a + \frac{1}{3} T_s^3 W_b$.
- Accelerometer bias variance: $T_s W_b$.
- Clock error variance: $T_s W_e$.

8-State Absolute Extended Kalman Filter (EKF)

Implemented in `xray_nav.py` and `route.ts`, this filter tracks the spacecraft's absolute coordinates relative to Earth's center:

$$x = \begin{bmatrix} r_x & r_y & r_z & v_x & v_y & v_z & b_{\text{clk}} & d_{\text{clk}} \end{bmatrix}^T$$

Absolute Orbital Dynamics

$$\frac{dr}{dt} = v, \quad \frac{dv}{dt} = a_{\text{gravity}}(r) + w_v, \quad \frac{db_{\text{clk}}}{dt} = d_{\text{clk}}, \quad \frac{dd_{\text{clk}}}{dt} = w_d$$

We model the gravity acceleration a_{gravity} by combining Keplerian two-body dynamics with the Earth's J_2 oblateness perturbation:

$$a_{\text{Kepler}} = -\frac{\mu}{r^3}$$

$$a_{J_2} = \frac{3}{2} J_2 \frac{\mu}{R_E^2 r^4} \begin{bmatrix} (5\frac{r_z^2}{r^2} - 1) \frac{r_x}{r} \\ (5\frac{r_z^2}{r^2} - 1) \frac{r_y}{r} \\ (5\frac{r_z^2}{r^2} - 3) \frac{r_z}{r} \end{bmatrix}$$

We propagate these non-linear equations forward using 4th-order Runge-Kutta (RK4) integration:

$$k_1 = f(x_n), \quad k_2 = f(x_n + \frac{\Delta t}{2} k_1), \quad k_3 = f(x_n + \frac{\Delta t}{2} k_2), \quad k_4 = f(x_n + \Delta t k_3)$$

$$x_{n+1} = x_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

Linearized Jacobians

To propagate the covariance matrix P, we calculate the Jacobian $F = \frac{\partial f}{\partial x}$ of the dynamics at each step. This requires the 3 x 3 gravity gradient matrix $G(r)$:

$$G(r) = -\frac{\mu}{r^3} I_3 + \frac{3\mu}{r^5} r r^T$$

$$F = \begin{bmatrix} 0_{3 \times 3} & I_3 & 0_{3 \times 1} & 0_{3 \times 1} \\ G(r) & 0_{3 \times 3} & 0_{3 \times 1} & 0_{3 \times 1} \\ 0_{1 \times 3} & 0_{1 \times 3} & 0 & 1 \\ 0_{1 \times 3} & 0_{1 \times 3} & 0 & 0 \end{bmatrix}$$

The state transition matrix is approximated as $\Phi \sim I + F \Delta t$, which is used to update the covariance:

$$P_{n+1} = \Phi P_n \Phi^T + Q$$

Standard Update vs. Joseph Form

In `kalman_filter.py`, we implement the Joseph form of the covariance update to ensure numerical stability:

$$P^{+} = (I - KH) P^{-} (I - KH)^T + K R K^T$$

This formulation preserves the symmetry and positive-definiteness of the covariance matrix, even in the presence of floating-point rounding errors.

12. Monte Carlo Simulation

Why It Is Used

Kalman filters and least-squares estimators rely on linear approximations of non-linear geometry and orbital mechanics. To verify that these estimators are unbiased and consistent under realistic operating conditions, we use Monte Carlo (MC) Simulation.

By running hundreds of independent trials with randomized initial states, sensor noise, and clock errors, we can analyze the statistical behavior of the navigation solution.

Noise Modeling

In our MC engine:

14. Initial Position Error: Sampled from the initial covariance: $r_0 \sim \mathcal{N}(r_{\text{true}}, P_0)$.
15. Timing Noise: Added to each pulsar delay measurement. The noise is modeled as zero-mean Gaussian: $\eta_i \sim \mathcal{N}(0, \sigma_i^2)$, where the standard deviation σ_i is derived from the Cramér-Rao Lower Bound.
16. Clock Bias: Sampled from a uniform distribution: $b \sim \text{Uniform}(-10 \mu\text{s}, +10 \mu\text{s})$, representing a realistic range of initial onboard clock errors.

Statistical Validation (Filter Consistency)

A filter is consistent if its estimated state covariance matrix P accurately reflects the actual errors. To evaluate this, we compare:

- Monte Carlo Standard Deviation (σ_{mc}): The empirical standard deviation of the navigation error across all MC runs:

$$\sigma_{\text{mc}, i}(t) = \sqrt{\frac{1}{M-1} \sum_{m=1}^M (x_{\text{est}, i}^{(m)}(t) - x_{\text{true}, i}^{(m)}(t))^2}$$

- Analytical Standard Deviation (σ_{filter}): The standard deviation computed by the filter:

$$\sigma_{\text{filter}, i}(t) = \sqrt{P_{ii}(t)}$$

If the filter is correctly tuned, the empirical error envelope σ_{mc} will match the analytical envelope σ_{filter} over time.

13. CRLB Analysis

Theoretical Background

The Cramér-Rao Lower Bound (CRLB) defines the absolute minimum variance that any unbiased estimator can achieve. For estimating a parameter vector θ , the estimator variance is bounded by the inverse of the Fisher Information Matrix $I(\theta)$:

$$\text{Cov}(\theta) \geq I(\theta)^{-1}$$

Fisher Information Integral I_p (Eq 4.11)

For pulsar phase estimation, the Fisher Information obtained from 1 second of observation depends on the pulse shape:

$$I_p = \int_0^1 \frac{1}{\lambda_b} \left(\frac{h'(\phi)}{h(\phi)} \right)^2 d\phi + \lambda_b \int_0^1 h(\phi) d\phi$$

- Pulse Sharpness ($h'(\phi)^2$): Steeper profiles (e.g., Crab or millisecond pulsars with sharp peaks) yield larger I_p , which reduces the timing bound.
- Signal strength (λ_s^2): Brighter pulsars yield higher information.
- Noise background (λ_b): High backgrounds (λ_b) dilute the signal and decrease I_p .

Timing and Distance Bounds

For absolute timing, the CRLB on the variance of the pulse delay t_d is:

$$\text{CRLB}(t_d) = \frac{1}{f_s^2} = \frac{1}{f_s^2} \cdot T_{\text{obs}} \cdot I_p$$

For relative navigation (measuring the delay between two detectors), the variance doubles because we subtract two independent phase estimates:

$$\text{CRLB}(t_d)_{\text{relative}} = \frac{2}{f_s^2} \cdot T_{\text{obs}} \cdot I_p$$

The lower bound on the position error standard deviation along the line of sight is:

$$\sigma_x = c \cdot \sqrt{\text{CRLB}(t_d)}$$

14. Dashboard and Navigation Lab

The Next.js frontend is a responsive, dark-mode web console designed for flight dynamics engineers and researchers.

```
+-----+
| [PulsarNav AI Console] [Region: Earth-Moon] [Pulsars: 6] [Solver: EKF] |
+-----+
|                                     |
|                                     | Mission Telemetry |
|                                     | ----- |
| 3D Space Scene | True Pos: [ 2.3k, 0.4k, 1k] |
| (Three.js)    | Est. Pos: [ 2.2k, 0.4k, 1k] |
|               | 3D Error: 0.084 km (Success)|
| * Spacecraft  | Clock Bias: 1.25 us |
| /             | +-----+
| /             | Active Pulsars |
| / * Earth     | ----- |
| /             | [x] J0613-0200 [x] J1713 |
| * Pulsar      | [x] J1909-3744 [x] J1744 |
```

+-----+-----+			
Error CDF Plot	Sensitivity Heatmap	Execution Log & CSV Export	
+-----+-----+			

Core Features

17. Interactive Controls: Users can configure simulation parameters (number of trials, active pulsars, timing noise level, mission region, and solver algorithm) via a control panel.
18. 3D Space Scene Visualizer: Renders Earth, the Moon, targeted pulsars, and the spacecraft trajectory.
 - Constructed using Three.js and React Three Fiber.
 - Integrates a 2D label projector with a priority-based box collision avoidance algorithm to prevent overlapping screen labels.
 - Hides secondary pulsar labels automatically when zoomed out ($r > 11.5$ units) to reduce visual clutter.
 - Includes real-time toggles to display or hide labels, reference axes, grids, and line-of-sight vectors.
19. Analytical Charts:
 - Error Summary: Displays median, mean, and 95th-percentile position errors.
 - Cumulative Distribution Function (CDF): Plots the probability distribution of position errors.
 - Sensitivity Heatmap: Displays position error as a function of timing noise and the number of active pulsars.
 - Histogram: Bins the 3D position errors to illustrate the shape of the error distribution.
20. Data Export: Provides one-click CSV download of the raw trial telemetry (true position, estimated position, timing delays, and coordinate errors).

15. Current Project Status

Completed Components

- Astronomical Parsers: Fully functional `.par`` and `.tim`` parsers in `parsing.py` that process TEMPO/TEMPO2 data and export catalog statistics.
- Coordinate Systems: Coordinate transformations between equatorial, ecliptic, and galactic frames, and unit-vector conversions.
- Barycentric Timing Model: Roemer delay models and $1/f^2$ plasma dispersion corrections.
- Photon Generator: Non-Homogeneous Poisson Process (NHPP) simulation using Newton-Raphson accumulated rate inversion.
- Epoch-Folding Engine: Phase binning, rate normalization, and velocity-error smearing analysis.
- Phase Delay Estimators: Cross-Correlation (FFT), Nonlinear Least Squares (NLS), and Maximum Likelihood (MLE) algorithms.
- Linear Solvers: Least Squares (LS) and Weighted Least Squares (WLS) coordinate estimation engines.
- Cramér-Rao Bounds: Fisher Information integrals, CRLB covariance limits, and Asymptotic Relative Efficiency (ARE) calculations.
- Linear Kalman Filter: 10-state error-state Kalman filter with Joseph-form updates.
- Web UI Dashboard: Fully responsive Next.js frontend with dark-mode styling, Three.js 3D space scene, and Recharts analytical plots.

Partially Completed Components

- High-Fidelity EKF Integration: Absolute-state 8-state EKF with gravity perturbation (J_2) and RK4 propagation. It is fully implemented and operational in the Next.js API route

(`app/api/navigation-lab/route.ts`) and prototype script (xray_nav.py), but it is not yet integrated into the main command-line pipeline launcher script (`run\_pipeline.py`).

- **Real-time Timing Residuals:** The dashboard UI contains a visual placeholder for streaming timing residuals, but it is currently fed by static mocked files rather than a live SSE (Server-Sent Events) or WebSocket data stream.

Pending Components

- **Covariance Ellipsoids:** Rendering of 3D error covariance ellipsoids around the spacecraft position in the Three.js scene.
- **Parallel Simulation Loop:** Optimizing the Monte Carlo engine to run trials in parallel using Python thread pools or Web Workers.

16. Future Roadmap

1. Navigation Lab Stabilization

Optimize the execution speed of larger Monte Carlo simulations (e.g., 5,000+ trials) on the Next.js backend by moving the simulation loop to a Web Worker thread, preventing main-thread blocking during interactive slider adjustments.

2. ATNF Catalog Integration

Integrate client-side API calls to query the Australia Telescope National Facility (ATNF) database in real time. This will allow users to dynamically add new pulsars to the simulation catalog and query their parameters.

3. High-Precision Ephemerides

Replace the Keplerian analytical Earth orbit model with the JPL DE440/DE441 Planetary Ephemeris, using barycentric positions computed from Chebyshev polynomials to reduce positioning errors from hundreds of meters to sub-meter levels.

4. Advanced EKF Formulations

Extend the 8-state absolute EKF to support range-rate measurements from inter-satellite links, enabling cooperative multi-spacecraft navigation (swarm navigation) around Mars or the Moon.

5. DSN vs. Pulsar Comparative Studies

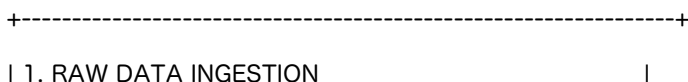
Develop a simulation dashboard that models DSN performance alongside XNAV, illustrating how XNAV outperforms DSN as the spacecraft travels further from Earth.

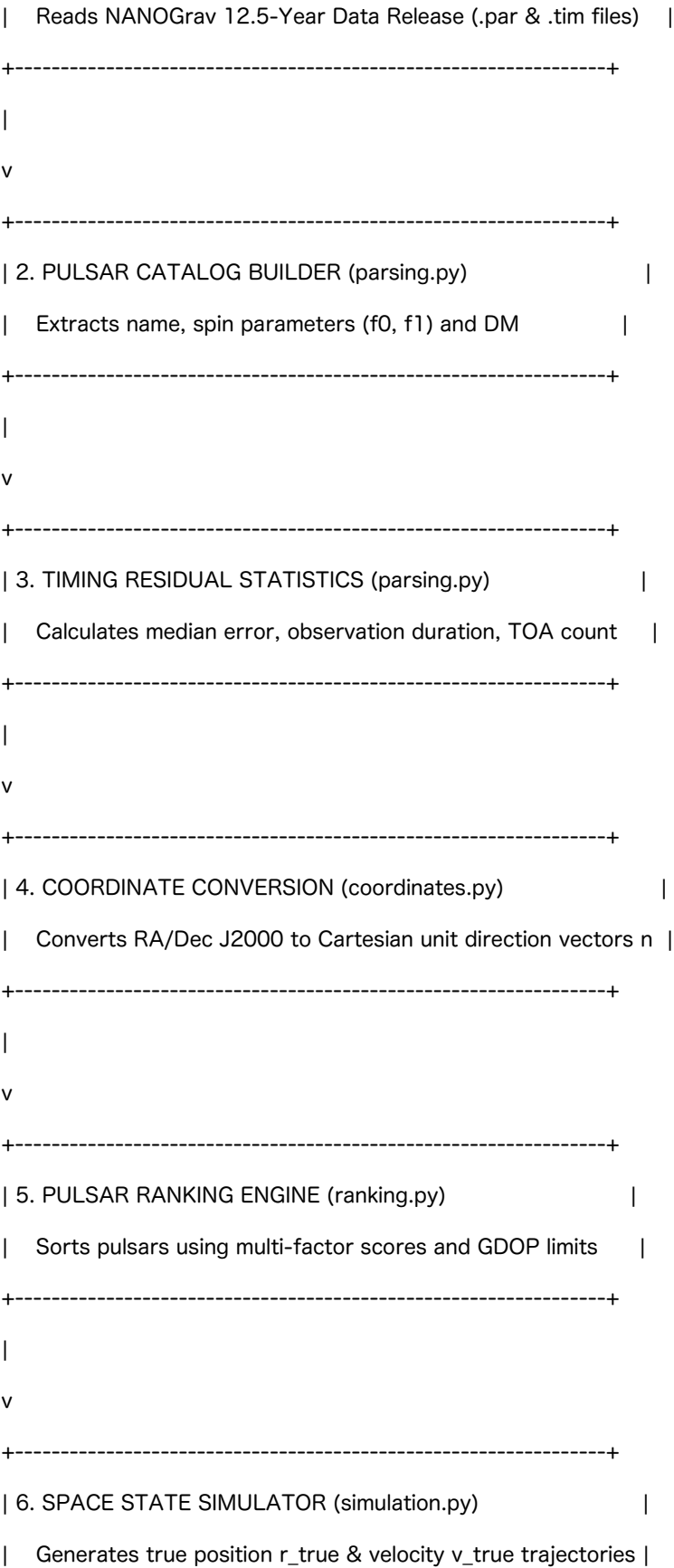
6. Hybrid Navigation Systems

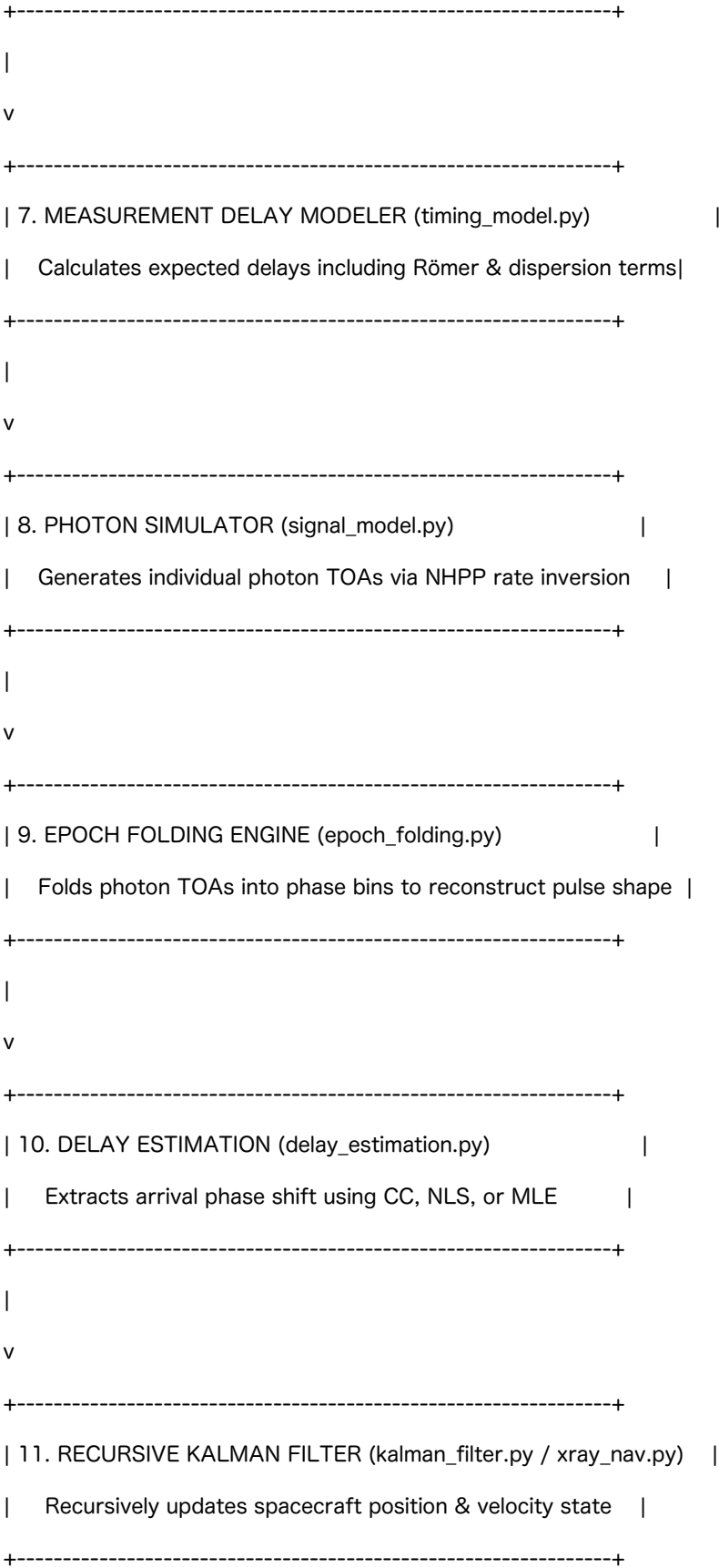
Implement a hybrid EKF that fuses radio tracking (from DSN/ground tracking) with X-ray pulsar observations. Fusing these measurements utilizes the strengths of both systems: DSN provides high accuracy near Earth, while XNAV bounds error growth during long cruises.

17. End-to-End Data Flow

The complete PulsarNav AI data pipeline maps raw astronomical data to user-facing dashboard telemetry:







v	
+-----+	
12. DASHBOARD VISUALIZATION (components/)	
Renders 3D orbital trajectory, error curves, and CDF plots	
+-----+	

Detailed Pipeline Stages

- 21. Raw Data Ingestion: Reads the raw narrowband `.par` and `.tim` ASCII files from the NANOGraV dataset.
- 22. Pulsar Catalog Builder: Parses coordinates, spin frequency, and dispersion parameters into a catalog DataFrame.
- 23. Timing Residual Statistics: Analyzes raw TOA timings to determine measurement errors and observation periods.
- 24. Coordinate Conversion: Converts Right Ascension and Declination angles into 3D unit vectors.
- 25. Pulsar Ranking Engine: Ranks pulsars based on signal stability, measurement count, and spatial separation to select the best navigation beacons.
- 26. Space State Simulator: Simulates orbital trajectories across Earth orbit, Earth-Moon, and deep space regions.
- 27. Measurement Delay Modeler: Computes expected delays relative to the SSB, incorporating solar system geometry and dispersion.
- 28. Photon Simulator: Generates realistic photon arrival times at the spacecraft using a Poisson rate model.
- 29. Epoch Folding Engine: Phase-folds the simulated photon TOAs to reconstruct the observed pulse profile.
- 30. Delay Estimation: Matches the folded profile to the template using CC, NLS, or MLE to determine the timing offset.
- 31. Recursive Kalman Filter: Fuses the timing offsets into the Kalman Filter state to estimate position, velocity, and clock bias.
- 32. Dashboard Visualization: Renders the true and estimated trajectories in the 3D space scene, alongside performance plots.

18. Mathematical Concepts Summary

This section consolidates all key mathematical equations implemented in the framework.

Equation Description	Formula	Variables & Meanings	Units	Implementing Module
Galactic to Cartesian	$x = \cos(b)\cos(l)$ $y = \cos(b)\sin(l)$ $z = \sin(b)$	l : Galactic longitude b : Galactic latitude	l, b : radians x, y, z : unitless	<code>`galactic_to_cartesian`</code> in <code>navigation_geometry.py</code>
Equatorial to Cartesian	$x = \cos(\delta)\cos(\alpha)$ $y = \cos(\delta)\sin(\alpha)$ $z = \sin(\delta)$	α : Right Ascension δ : Declination	α, δ : radians x, y, z : unitless	<code>`sky_to_unit_vector`</code> in <code>coordinates.py</code>
Römer Delay	$\Delta t_R = \frac{r}{c} \cdot n$	r : Position relative to origin n : Pulsar unit direction vector c :	r : km n : unitless Δt_R : seconds	<code>`roemer_delay_ssb_s`</code> in <code>timing_model.py</code>

Equation Description	Formula	Variables & Meanings	Units	Implementing Module
		Speed of light		
Dispersion Delay	$\Delta t_{DM} = \frac{4.148808}{c} \times 10^3 \cdot DM f^2$	DM: Dispersion Measure f: Observing frequency	DM: pc/cm ³ f: MHz Δt_{DM} : seconds	<code>`dispersion_delay_s`</code> in <code>timing_model.py</code>
Poisson Pulse Rate	$\lambda(t) = \lambda_b + \lambda_s h(\phi(t))$	λ_b : Background photon rate λ_s : Source photon rate h: Normalized profile	rates: ph/s ϕ : cycles	<code>`lambda_const`</code> in <code>signal_model.py</code>
Epoch-Folding Rate	$\bar{\lambda}(t_i) = \frac{C_i}{N_b T_{obs}}$	C_i : Count in bin i N_b : Total number of bins T_{obs} : Observation time	rates: ph/s T_{obs} : seconds	<code>`epoch_fold`</code> in <code>epoch_folding.py</code>
Velocity Limit	$\Delta v_{max} = \frac{c}{N_b} \cdot N_p$	N_b : Bins per period N_p : Number of folded periods	v: m/s c: m/s	<code>`velocity_error_tolerance`</code> in <code>epoch_folding.py</code>
Fisher Information	$I_p = \int_0^1 \frac{[\lambda_s h'(\phi)]^2}{\lambda_b + \lambda_s h(\phi)} d\phi$	h': Pulse profile derivative	I_p : unitless	<code>`fisher_integral_ip`</code> in <code>crlb.py</code>
Delay CRLB	$CRLB(t_d) = \frac{2}{f_s^2 T_{obs} I_p}$	f_s : Pulsar spin frequency	variance: s ²	<code>`crlb_pulse_delay`</code> in <code>crlb.py</code>
NLS Cost Function	$J(\phi) = \sum_{i=1}^{N_b} (\bar{\lambda}_i - \lambda_i(\phi))^2$	$\bar{\lambda}$: Empirical rate $\lambda(\phi)$: Shifted template	J: (ph/s) ²	<code>`estimate_phase`</code> in <code>delay_estimation.py</code>
MLE Log-Likelihood	$\Psi(\phi) = \sum_{i=1}^{N_{ph}} \ln(\lambda_b + \lambda_s h(f_o t_i + \phi))$	t_i : Raw photon arrival times	Ψ : unitless	<code>`estimate_phase`</code> in <code>delay_estimation.py</code>
Linearized Dynamics	$\Phi \sim I + F \Delta t$	F: Dynamics Jacobian	matrices: unitless	<code>`state_transition_Phi`</code> in <code>navigation_geometry.py</code>
Covariance Update	$P^+ = (I - KH) P^- (I - KH)^T + K R K^T$	K: Kalman gain H: Measurement matrix R: Noise covariance	matrices: standard SI units	<code>`update`</code> in <code>kalman_filter.py</code>
Gravity Gradient	$G(r) = -\frac{\mu}{r^3} I + \frac{3\mu}{r^5} r r^T$	μ : Earth gravity parameter r: Position vector	G: s ⁻²	<code>`_gravity_gradient`</code> in <code>xray_nav.py</code>
PDOP	$PDOP = \sqrt{\text{Tr}(\Gamma^{-1})}$	Γ : Pulsar direction matrix	DOP: unitless	<code>`position_dilution`</code> in <code>navigation_geometry.py</code>

Equation Description	Formula	Variables & Meanings	Units	Implementing Module
	Γ^{-1}			py

19. File-by-File Mapping

This catalog defines the purpose, inputs, outputs, concepts, and dependencies of the core Python scientific files:

1. coordinates.py

- Purpose: Handles transformations between astronomical coordinate systems and unit vector generation.
- Inputs: Sexagesimal Right Ascension and Declination strings, or ecliptic angles (degrees).
- Outputs: 3D Cartesian unit vectors $[x, y, z]$ and `SkyCoord` dataclasses.
- Concepts Implemented: Ecliptic-to-equatorial coordinate rotation, sexagesimal angle parsing (conversion of hours/minutes/seconds to degrees), and sphere-to-Cartesian mappings.
- Dependencies: `numpy`, `math`, `re`, `dataclasses`.

2. timing_model.py

- Purpose: Implements time propagation models and relativistic/medium corrections.
- Inputs: Spacecraft coordinates (km), pulsar line-of-sight unit vectors, Modified Julian Date (MJD), Dispersion Measure (DM), and frequency (MHz).
- Outputs: Timing delays (seconds).
- Concepts Implemented: Earth-Moon Barycenter analytical orbital dynamics, relative Römer delay ($\Delta t_R = r \cdot n/c$), and interstellar plasma dispersion delay ($1/f^2$).
- Dependencies: `numpy`.

3. signal_model.py

- Purpose: Models X-ray photon arrivals at a detector using a Poisson model.
- Inputs: Background flux λ_b , source flux λ_s , spin frequency f_s , initial phase ϕ_0 , and observation time T_{obs} .
- Outputs: Non-Homogeneous Poisson Process (NHPP) simulated photon Times of Arrival (TOAs) and evaluated rates.
- Concepts Implemented: Period-folded Gaussian-sum templates, time-varying observed frequencies (Doppler shifting), accumulated rate integration ($\Lambda(t)$), and Newton-Raphson inversion.
- Dependencies: `numpy`, `scipy.interpolate.interp1d`, `pulsar\_nav/pulsar\_catalog.py`.

4. epoch_folding.py

- Purpose: Implements phase folding and velocity-error smearing analysis.
- Inputs: Photon TOA arrays (seconds), folding frequency f_o (Hz), bin resolution N_b , and velocity error Δv (m/s).
- Outputs: Empirical profile rates and noise variances.
- Concepts Implemented: Phase folding, histogram normalization, Poisson bin variance, and phase-smearing limits.
- Dependencies: `numpy`, `pulsar\_nav/pulsar\_catalog.py`, `pulsar\_nav/signal\_model.py`.

5. delay_estimation.py

- Purpose: Estimates the arrival phase difference of a pulse at a detector.

- Inputs: Folded empirical rates, true profile templates, raw photon TOAs, background/source rates, and grid sizes.
- Outputs: Phase estimates ϕ_0 (cycles), timing delays (seconds), and cost functions.
- Concepts Implemented: Circular Cross-Correlation (FFT), parabolic peak interpolation, Nonlinear Least Squares (NLS) grid search, and Maximum Likelihood Estimation (MLE) on raw TOAs.
- Dependencies: ``numpy``, ``scipy.optimize.minimize_scalar``, ``pulsar_nav/signal_model.py``, ``pulsar_nav/epoch_folding.py``.

6. crlb.py

- Purpose: Computes the fundamental statistical limits of XNAV timing accuracy.
- Inputs: Profile templates, background/source rates, observation duration T_{obs} , and observing frequencies.
- Outputs: Fisher Information integrals, CRLB variance bounds, and Asymptotic Relative Efficiency (ARE) ratios.
- Concepts Implemented: Numerical quadrature of Fisher integrals, joint parameter CRLB matrices (phase + frequency), and Asymptotic Relative Efficiency (information loss due to binning).
- Dependencies: ``numpy``, ``pulsar_nav/pulsar_catalog.py``, ``pulsar_nav/signal_model.py``.

7. navigation_geometry.py

- Purpose: Implements state-space structures, dynamics matrices, and observability checks.
- Inputs: Selected pulsar lists and sampling intervals.
- Outputs: System dynamics matrix F , state transition matrix Φ , measurement matrix H , and process noise covariance Q .
- Concepts Implemented: State-space modeling, nilpotent matrix transitions ($F^3=0$), discrete covariance integration, rank-based system observability checks, and GDOP/PDOP.
- Dependencies: ``numpy``, ``math``, ``pulsar_nav/pulsar_catalog.py``.

8. kalman_filter.py

- Purpose: Implements the 10-state linear error-state Kalman Filter (L-ESKF).
- Inputs: Simulated or real measurement delays, measurement variances, sampling intervals, and true states.
- Outputs: Estimated state corrections (position error, velocity error, accelerometer bias, clock bias), state covariances, and innovation records.
- Concepts Implemented: Linear Kalman Filter prediction and update cycles, Joseph-form covariance updates, and Monte Carlo filter consistency verification.
- Dependencies: ``numpy``, ``pulsar_nav/pulsar_catalog.py``, ``pulsar_nav/navigation_geometry.py``.

9. xray_nav.py

- Purpose: High-fidelity absolute Extended Kalman Filter (EKF) engine for Earth orbiters.
- Inputs: Trajectory steps, initial states, process/measurement noise levels, and pulsar line-of-sight parameters.
- Outputs: Absolute position, velocity, and clock bias estimates, and covariances.
- Concepts Implemented: Absolute EKF, Keplerian orbit dynamics, Earth J_2 oblateness gravity perturbations, gravity gradient matrices, and 4th-order Runge-Kutta numerical integration.
- Dependencies: ``numpy``, ``pandas``, ``scipy.optimize.minimize_scalar``, ``pulsar_nav/config.py``.

10. navigation_engine.py

- Purpose: Runs batch least-squares Monte Carlo simulations.
- Inputs: Ranked pulsar CSV paths, noise levels, pulsar counts, and trial quantities.
- Outputs: Computed positions, coordinate errors, statistical summaries, and SVG error plots.

- Concepts Implemented: Trajectory simulation, Gaussian timing noise injection, clock bias simulation, WLS solvers, error percentiles, and SVG plotting.
- Dependencies: `numpy`, `pandas`, `pathlib`, `pulsar\_nav/config.py`, `pulsar\_nav/timing\_model.py`.

20. Research Contribution

The PulsarNav AI framework contributes to deep-space navigation research in several ways:

Scientific Contribution

The framework provides an independent validation platform for the mathematical models of X-ray pulsar emission and propagation delay. By implementing Cross-Correlation, Nonlinear Least Squares, and Maximum Likelihood estimators, the codebase allows researchers to analyze the trade-off between timing accuracy and computational complexity under realistic noise profiles.

Furthermore, the Asymptotic Relative Efficiency (ARE) analysis quantifies the information loss that occurs when continuous photon times of arrival are binned into discrete phase histograms.

Engineering Contribution

Integrating high-fidelity Python scientific calculations with a Next.js web application demonstrates a modern approach to mission-control design. The Next.js API route ([route.ts](#)) implements equivalent mathematical formulations of coordinate rotations, Keplerian orbital dynamics, J_2 gravity perturbations, and EKF algorithms in TypeScript, enabling real-time interactive simulations in the web visualizer.

The 60 fps Three.js rendering engine features a priority-based label projection loop, showing that complex scientific visualizers can run smoothly in standard web browsers.

Autonomous Navigation Significance

By demonstrating that a spacecraft can determine its position and velocity using millisecond pulsars, this framework supports the transition toward fully autonomous deep-space probes. An onboard XNAV system enables:

33. Self-Governing Cruise Phases: Spacecraft can cruise through interplanetary space and calculate their trajectories without requiring routine ground stations support.
34. Autonomous Trajectory Corrections: Real-time onboard state estimation enables immediate orbital corrections, reducing risk during critical mission phases such as outer-planet insertions or flybys.
35. Emergency Navigation Fallbacks: If communication with Earth is lost, the spacecraft can use XNAV as a fail-safe navigation system to maintain its course.

Relevance to SAC ISRO

As the Space Applications Centre (SAC) ISRO plans ambitious future missions—including cislunar orbiters, Martian landing platforms, and deep-space probes to the outer solar system—autonomous navigation is critical. Researching XNAV algorithms supports ISRO's goals of developing self-reliant space technologies.

Fusing timing measurements from onboard X-ray spectrometers with terrestrial tracking systems will help secure spacecraft autonomy and ensure the success of India's future deep-space exploration programs.

17. Deliverable-Oriented Achievement Matrix

Achievement	Implemented Evidence	Validation / Output
GPS-independent spacecraft navigation framework	Python modules and Next.js Navigation Lab solve spacecraft position from pulsar timing delays.	Monte Carlo trials, dashboard telemetry, CSV exports.
NANOGrav 12.5-year dataset integration	TEMPO/TEMPO2 PAR/TIM parsing pipeline and catalog/statistics generation.	Catalog, unit-vector, TOA database, statistics, and ranked pulsar CSVs.
NHPP signal modeling	RateFunction and photon TOA generation use non-homogeneous Poisson process logic.	Delay-estimator tests and CRLB comparison surfaces.
Epoch folding	Photon TOAs are folded into phase histograms with velocity-error tolerance checks.	Profile recovery, phase histograms, and estimator inputs.
Delay estimation	CC, NLS, and MLE phase estimators are implemented for pulse-delay recovery.	Pulse delay tests and compatibility wrapper.
Navigation geometry	Direction matrices, measurement matrices, GDOP, and observability utilities.	Pulsar set optimization and rank/condition diagnostics.
LS/WLS/EKF estimation	Linear position solvers, 10-state error-state KF, and 8-state absolute EKF.	Navigation Lab API, trajectory visualization, and unit tests.
3D visualization	React Three Fiber and Three.js spacecraft, Earth/Moon, pulsar, line-of-sight, and trajectory rendering.	Browser-tested trajectory playback and label collision handling.
Comparative navigation lab	DSN, XNAV, and Hybrid EKF simulator in TypeScript.	Generated comparison report and mission KPI tables.

18. Submission Roadmap

36. Phase 1: Freeze this design document as the technical baseline for project submission.
37. Phase 2: Insert final screenshots, generated plots, and Monte Carlo result tables from the latest run.
38. Phase 3: Add appendix-level source listings only where required by academic submission rules.
39. Phase 4: Convert the executive summary into an oral-defense abstract and presentation script.

Appendix A. Equation Implementation Map

Book Equation Map — "Navigation in Space by X-ray Pulsars" (2011)

Authors: Amir Abbas Emadzadeh, Jason Lee Speyer

Publisher: Springer, New York

DOI: 10.1007/978-1-4419-8017-5

Chapter 3: Signal Modeling

3.1 Geometric Range

Eq.	Description	Formula	Variables	Units	Implementation
3.1	Projected distance between spacecraft	$\Delta d = n \cdot \Delta x = c \cdot t_d$	$\hat{n}$: unit direction vector to pulsar, Δx : relative position, c : speed of light, t_d : time delay	m, m, m/s, s	<code>`pulsar_nav/signal_model.py`</code>

3.2 Non-Homogeneous Poisson Process (NHPP)

Eq.	Description	Formula	Variables	Units	Implementation
3.2	Ordered TOA sequence	$t_0 \leq t_1 < t_2 < \dots < t_M \leq t_f$	t_i : TOA of i th photon, M : total photon count	s	<code>`pulsar_nav/signal_model.py`</code>
3.3	Point process definition	$N_t = \max n, t_n \leq t$	N_t : number of photons in $(0, t)$	count	<code>`pulsar_nav/signal_model.py`</code>
3.4	Single photon probability	$P(N_{t+\Delta t} - N_t = 1) = \lambda(t)\Delta t$	$\lambda(t)$: rate function	ph/s	<code>`pulsar_nav/signal_model.py`</code>
3.5	No simultaneous photons	$P(N_{t+\Delta t} - N_t \geq 2) = 0$	—	—	<code>`pulsar_nav/signal_model.py`</code>
3.7	Poisson distribution for count	$P(N_t = k) = \left[\int_0^t \lambda(\xi) d\xi \right]^k \exp(-\int_0^t \lambda(\xi) d\xi) / k!$	k : photon count	count	<code>`pulsar_nav/signal_model.py`</code>
3.8	Mean and variance of N_t	$E[N_t] = \text{var}[N_t] = \int_0^t \lambda(\xi) d\xi \triangleq \Lambda(t)$	$\Lambda(t)$: integrated rate	count	<code>`pulsar_nav/signal_model.py`</code>
3.9	Interval Poisson distribution	$P(N_t - N_s = k) = \left[\int_s^t \lambda(\xi) d\xi \right]^k \exp(-\int_s^t \lambda(\xi) d\xi) / k!$	—	count	<code>`pulsar_nav/signal_model.py`</code>
3.10	Joint TOA PDF	$p(t_i, M) = e^{-\Lambda}$	Λ : integrated	—	<code>`pulsar_nav/</code>

Eq.	Description	Formula	Variables	Units	Implementation
	(Theorem 3.1)	$\prod_i \lambda(t_i)$ for $M \geq 1$	rate over $[t_0, t_f]$		signal_model.py
3.11	Integrated rate	$\Lambda \triangleq \Lambda(t_f) - \Lambda(t_0)$	—	count	pulsar_nav/ signal_model.py

3.3 X-ray Pulsar Signal (Rate Function)

Eq.	Description	Formula	Variables	Units	Implementation
3.18	Overall rate function	$\lambda(t) = \lambda_b + \lambda_s \cdot h(\varphi_{\text{det}}(t))$	λ_b : background rate, λ_s : source rate, $h(\varphi)$: normalized profile	ph/s	pulsar_nav/ signal_model.py
3.19	Background rate	$\lambda_b = b \cdot \eta \cdot A$	b : background rate density, η : efficiency, A : area	ph/s	pulsar_nav/ signal_model.py
3.20	Source rate	$\lambda_s = s \cdot \eta \cdot A$	s : source flux	ph/s	pulsar_nav/ signal_model.py
3.21	Detected phase	$\varphi_{\text{det}}(t) = \varphi_0 + \int_{t_0}^t f_o(\tau) d\tau$	φ_0 : initial phase, f_o : observed frequency	cycle	pulsar_nav/ signal_model.py
3.22	Observed frequency decomposition	$f_o(t) = f_s + f_d(t)$	f_s : source freq, f_d : Doppler shift	Hz	pulsar_nav/ signal_model.py
3.23	Doppler frequency	$f_d(t) = f_s \cdot v(t) / c$	$v(t)$: detector velocity	Hz	pulsar_nav/ signal_model.py
3.24	Full detected phase	$\varphi_{\text{det}}(t) = \varphi_0 + f_s(t - t_0) + \int_{t_0}^t f_d(\tau) d\tau$	—	cycle	pulsar_nav/ signal_model.py

3.4 Constant-Frequency Model

Eq.	Description	Formula	Variables	Units	Implementation
3.25	Constant-frequency phase	$\varphi_{\text{det}}(t) = \varphi_0 + (t - t_0) \cdot f_o$	f_o : observed constant freq	cycle	pulsar_nav/ signal_model.py
3.26	Observed frequency	$f_o = (1 + v/c) \cdot f_s$	v : constant velocity	Hz	pulsar_nav/ signal_model.py
3.27	Constant-	$\lambda(t; \varphi_0, f_o) =$	—	ph/s	pulsar_nav/

Eq.	Description	Formula	Variables	Units	Implementation
	frequency rate function	$\lambda_b + \lambda_s \cdot h(\varphi_0 + (t - t_0) \cdot f_o)$			signal_model.py

3.5 Time-Dependent-Frequency Model

Eq.	Description	Formula	Variables	Units	Implementation
3.28	Doppler phase	$\varphi_d(t) = \int_{t_0}^t f_d(\tau) d\tau$	—	cycle	pulsar_nav/ signal_model.py
3.29	Time-dependent detected phase	$\varphi_{det}(t) = \varphi_0 + (t - t_0) \cdot f_s + \varphi_d(t)$	—	cycle	pulsar_nav/ signal_model.py
3.30	Time-dependent rate function	$\lambda(t; \varphi_0, v(t)) = \lambda_b + \lambda_s \cdot h(\varphi_0 + (t - t_0) \cdot f_s + \varphi_d(t))$	—	ph/s	pulsar_nav/ signal_model.py

3.6 Epoch Folding

Eq.	Description	Formula	Variables	Units	Implementation
3.38	Poisson bin count statistics	$E[c(t_i)] = \text{var}[c(t_i)] = \lambda(t_i) \cdot T_b$	$c(t_i)$: count in i th bin, T_b : bin width	count	pulsar_nav/ epoch_folding.py
3.39	Empirical rate function	$\lambda(t_i) = (1/(N_p \cdot T_b)) \cdot \sum_j c_j(t_i)$	N_p : number of periods	ph/s	pulsar_nav/ epoch_folding.py
3.40	Empirical = true + noise (Thm 3.2)	$\lambda(t_i) = \lambda(t_i) + n(t_i)$	n : epoch folding noise	ph/s	pulsar_nav/ epoch_folding.py
3.41	Noise mean	$E[n(t_i)] = 0$	—	—	pulsar_nav/ epoch_folding.py
3.42	Noise variance	$\text{var}[n(t_i)] = (N_b / T_{obs}) \cdot \lambda(t_i)$	N_b : num bins, T_{obs} : obs time	(ph/s) ²	pulsar_nav/ epoch_folding.py

3.7 Epoch Folding with Velocity Errors

Eq.	Description	Formula	Variables	Units	Implementation
3.59	Frequency shift from velocity error	$\Delta f_o = f_s \cdot \Delta v / c$	Δv : velocity error	Hz	pulsar_nav/ epoch_folding.py

Eq.	Description	Formula	Variables	Units	Implementation
3.60	Period change	$\Delta P = -\Delta f_o / f_o^2$	—	s	`pulsar_nav/ epoch_folding.py`
3.61	Period change simplified	$\Delta P = -(f_s / (c \cdot f_o^2)) \cdot \Delta v$	—	s	`pulsar_nav/ epoch_folding.py`
3.66	Phase change	$\Delta \phi \approx -(1/c) \cdot \Delta v$	—	cycle	`pulsar_nav/ epoch_folding.py`
3.68	Empirical rate with velocity error	$\lambda(\tilde{t}_i) = (1/\tilde{N}_p) \sum_j \lambda(t_i; \varphi_0 + (j-1)\Delta\phi) + n(\tilde{t}_i)$	$\tilde{N}_p$: perturbed period count	ph/s	`pulsar_nav/ epoch_folding.py`
3.72	Velocity error tolerance bound	$\Delta v < c / (N_b \cdot \tilde{N}_p)$	—	m/s	`pulsar_nav/ epoch_folding.py`

3.8 TOA Simulation (Algorithm 3.1)

Eq.	Description	Formula	Variables	Units	Implementation	
3.76	Inter-arrival CDF	$P(Z > z)$	$t_n = t = \exp(-(\Lambda(t+z) - \Lambda(t)))$	Z : inter-arrival time	—	`pulsar_nav/ signal_model.py`
3.80	TOA generation via inversion	$Z = -t + \Lambda^{-1}(\Lambda(t) - \ln U)$	$U \sim \text{Uniform}(0,1)$	s	`pulsar_nav/ signal_model.py`	
3.86	Recursive TOA generation	$t_{n+1} = \Lambda^{-1}(\Lambda(t_n) + E)$, $E \sim \text{Exp}(1)$	—	s	`pulsar_nav/ signal_model.py`	
Alg 3.1	TOA Simulation Algorithm	See book p. 42	—	—	`pulsar_nav/ signal_model.py`	

Chapter 4: Pulse Delay Estimation

4.1 Pulse Delay Formulation

Eq.	Description	Formula	Variables	Units	Implementation
4.1	Rate function on detector 1	$\lambda_1(t; \varphi_i) = \lambda_b + \lambda_s \cdot h(\varphi_i + (t - t_0) \cdot f_i)$	f_i : observed freq at det 1	ph/s	`pulsar_nav/ delay_estimation.py`
4.2	Rate function on detector 2	$\lambda_2(t; \varphi_i) = \lambda_b + \lambda_s \cdot h(\varphi_i + (t - t_0$	t_e : clock differential, t_x :	ph/s	`pulsar_nav/ delay_estimation.py`

Eq.	Description	Formula	Variables	Units	Implementation
		$-t_e - t_x \cdot f_2$	true time delay		<code>py`</code>
4.3	Phase-delay relation	$\varphi_2 \triangleq \varphi_1 - f_2 \cdot t_d$, where $t_d = t_e + t_x$	t_d : pulse delay	s	<code>pulsar_nav/` delay_estimation.py`</code>
4.43	Pulse delay estimate	$\hat{t}_d = (\hat{\phi}_1 - \hat{\phi}_2) / \hat{f}_2$	—	s	<code>pulsar_nav/` delay_estimation.py`</code>

4.2 Cramér–Rao Lower Bound (CRLB)

Eq.	Description	Formula	Variables	Units	Implementation
4.6	CRLB inequality	$\text{cov}(\theta) \geq I^{-1}(\theta)$	$I(\theta)$: Fisher matrix	—	<code>pulsar_nav/` crlb.py`</code>
4.7	Fisher matrix element	$[I(\theta)]_{ij} = -E[\partial^2 / \partial \theta_i \partial \theta_j \ln p(x; \theta)]$	—	—	<code>pulsar_nav/` crlb.py`</code>
4.10	Fisher matrix for (φ_o, f_o)	$I(\theta) = I_p / 6 \cdot$ [[$6T_{\text{obs}}$, $3T_{\text{obs}}^2$], [$3T_{\text{obs}}^2$, $2T_{\text{obs}}^3$]]	—	—	<code>pulsar_nav/` crlb.py`</code>
4.11	Fisher integral I_p	$I_p = \int_o^1$ [$\lambda_s \cdot h'(\varphi)^2 / [\lambda_b$ + $\lambda_s \cdot h(\varphi)] d\varphi$	—	—	<code>pulsar_nav/` crlb.py`</code>
4.12	CRLB matrix	$\text{CRLB}(\theta) =$ ($2/I_p$) · [[$2/T_{\text{obs}}$, $-3/T_{\text{obs}}^2$], [$-3/T_{\text{obs}}^2$, $6/T_{\text{obs}}^3$]]	—	—	<code>pulsar_nav/` crlb.py`</code>
4.22	Fisher element integral form	$I_{ij} = \int_{-t_0}^t f$ ($\partial \lambda / \partial \theta_i$)($\partial \lambda / \partial \theta_j$) / λ dt	—	—	<code>pulsar_nav/` crlb.py`</code>
4.42	CRLB for φ_o (known f_o)	$\text{CRLB}(\varphi_o) =$ $1 / (T_{\text{obs}} \cdot I_p)$	—	cycle ²	<code>pulsar_nav/` crlb.py`</code>
4.44	CRLB for pulse delay t_d	$\text{CRLB}(t_d) = 2 / (f_2^2$ · $T_{\text{obs}} \cdot I_p)$	—	s ²	<code>pulsar_nav/` crlb.py`</code>

Chapter 5: Pulse Delay Estimation Using Epoch Folding

5.1 Pulse Delay from Phase Estimates

Eq.	Description	Formula	Variables	Units	Implementation
5.1	Pulse delay from phases	$\hat{t}_d = (\hat{\phi}_1 - \hat{\phi}_2) / f_2$ (same as 4.43)	—	s	<code>`pulsar_nav/` <code>delay_estimation.`</code> <code>py`</code></code>

5.2 Cross Correlation (CC) Estimator

Eq.	Description	Formula	Variables	Units	Implementation
5.7	CC phase estimator	$\hat{\phi}_j = -\text{argmax}_{\psi} R_j(\psi)$	—	cycle	<code>`pulsar_nav/` <code>delay_estimation.`</code> <code>py`</code></code>
5.8	CC function (continuous)	$R_j(\psi) = \int \lambda^*(t) \cdot \lambda_j(t; \psi) dt$	—	—	<code>`pulsar_nav/` <code>delay_estimation.`</code> <code>py`</code></code>
5.9	CC estimator variance	$\text{var}[\hat{t}_d] = 2 \cdot \int_0^1 (\lambda_{-b} + \lambda_{-s} \cdot h(\varphi)) \cdot [\lambda_{-s} \cdot h'(\varphi)]^2 d\varphi / [f_2^2 \cdot T_{\text{obs}} \cdot (\int_0^1 [\lambda_{-s} \cdot h'(\varphi)]^2 d\varphi)^2]$	—	s ²	<code>`pulsar_nav/` <code>delay_estimation.`</code> <code>py`</code></code>
5.72	Discrete CC function	$R_D(\psi) = (1/N_b) \sum_k x_1(kT_b) \cdot x_2^*(kT_b; \psi)$	—	—	<code>`pulsar_nav/` <code>delay_estimation.`</code> <code>py`</code></code>
5.73	Fourier-domain CC	$R_D(\psi) = F^{-1} X_1(f_k) \cdot X_2^*(f_k)$	—	—	<code>`pulsar_nav/` <code>delay_estimation.`</code> <code>py`</code></code>
5.76	Parabolic interpolation	$\hat{\phi} \approx k_m \cdot T_s - \frac{1}{2} \cdot [R_D(k_m \cdot T_s + T_s) - R_D(k_m \cdot T_s - T_s)] / [R_D(k_m \cdot T_s + T_s) - 2R_D(k_m \cdot T_s) + R_D(k_m \cdot T_s - T_s)]$	—	cycle	<code>`pulsar_nav/` <code>delay_estimation.`</code> <code>py`</code></code>

5.3 Nonlinear Least Squares (NLS) Estimator

Eq.	Description	Formula	Variables	Units	Implementation
5.47	NLS cost function	$J(\varphi_j) = \sum_i (\lambda_j(t_i) - \lambda(t_i; \varphi_j))^2$	—	—	<code>`pulsar_nav/` <code>delay_estimation.`</code> <code>py`</code></code>

Eq.	Description	Formula	Variables	Units	Implementation
5.48	NLS estimator	$\hat{\phi}_j = \underset{\phi_j \in (0,1)}{\operatorname{argmin}} J(\phi_j)$	—	cycle	<code>`pulsar_nav/` <code>delay_estimation.py`</code></code>
5.65	NLS variance (discrete)	$\operatorname{var}[\hat{\phi}_j] = (N_b/T_{\text{obs}}) \cdot \sum \lambda(t_i; \phi_o) \cdot \lambda'^2(t_i; \phi_o) / (\sum \lambda'^2(t_i; \phi_o))^2$	—	cycle ²	<code>`pulsar_nav/` <code>delay_estimation.py`</code></code>
5.66	NLS variance (continuous)	$\operatorname{var}[\hat{\phi}_j] = \int_0^1 (\lambda_b + \lambda_s \cdot h(\phi)) \cdot [\lambda_s \cdot h'(\phi)]^2 d\phi / [T_{\text{obs}} \cdot (\int_0^1 [\lambda_s \cdot h'(\phi)]^2 d\phi)^2]$	—	cycle ²	<code>`pulsar_nav/` <code>delay_estimation.py`</code></code>

5.4 Asymptotic Relative Efficiency

Eq.	Description	Formula	Variables	Units	Implementation
5.39	ARE (CC/NLS vs CRLB)	$\operatorname{var}[\hat{t}_d] / \operatorname{CRLB}(t_d) = \int g_1 \cdot \int g_3 / (\int g_2)^2$	g_1, g_2, g_3 : see 5.40-5.42	—	<code>`pulsar_nav/` <code>crlb.py`</code></code>
5.40	g_1 definition	$g_1(\phi) = (\lambda_b + \lambda_s \cdot h(\phi)) \cdot [\lambda_s \cdot h'(\phi)]^2$	—	—	<code>`pulsar_nav/` <code>crlb.py`</code></code>
5.41	g_2 definition	$g_2(\phi) = [\lambda_s \cdot h'(\phi)]^2$	—	—	<code>`pulsar_nav/` <code>crlb.py`</code></code>
5.42	g_3 definition	$g_3(\phi) = [\lambda_s \cdot h'(\phi)]^2 / (\lambda_b + \lambda_s \cdot h(\phi))$	—	—	<code>`pulsar_nav/` <code>crlb.py`</code></code>

Chapter 6: Pulse Delay Estimation via Direct Use of TOAs

6.1 Maximum-Likelihood Estimator (MLE)

Eq.	Description	Formula	Variables	Units	Implementation
6.1	TOA likelihood function	$p(t^{(k)}_i; \phi_k, f_k) = e^{-\Lambda(\phi_k, f_k)} \cdot \prod_i \lambda_k(t^{(k)}_i; \phi_k, f_k)$	—	—	<code>`pulsar_nav/` <code>delay_estimation.py`</code></code>
6.2	Integrated rate for MLE	$\Lambda(\phi_k, f_k) = \int_{t_0}^{\infty} \lambda_k(t; \phi_k, f_k) dt$	—	count	<code>`pulsar_nav/` <code>delay_estimation.py`</code></code>

Eq.	Description	Formula	Variables	Units	Implementation
		$\varphi_k, f_k) dt$			py`
6.3	Log-likelihood function (full)	$LLF(\varphi_k, f_k) = \sum_i \ln(\lambda_k(t_i; \varphi_k, f_k)) - \Lambda(\varphi_k, f_k)$	—	—	`pulsar_nav/ delay_estimation. py`
6.5	Simplified ML cost function	$\Psi(\varphi_k, f_k) = \sum_i \ln(\lambda_k(t_i; \varphi_k, f_k))$	(drops Λ for long T_{obs})	—	`pulsar_nav/ delay_estimation. py`
6.6	Joint MLE optimization	$(\hat{\varphi}_k, \hat{f}_k) = \operatorname{argmax}_{\varphi_k, f_k} \Psi(\varphi_k, f_k)$	—	—	`pulsar_nav/ delay_estimation. py`
6.18	Phase-only MLE (known velocity)	$\hat{\varphi}_k = \operatorname{argmax}_{\varphi_k \in (0, 1)} \Psi(\varphi_k)$	—	cycle	`pulsar_nav/ delay_estimation. py`
6.20	Newton-Raphson iteration	$\theta^{(k+1)} = \theta^{(k)} - [\partial^2 / \partial \theta \partial \theta^T \ln p]^{-1} \cdot \partial / \partial \theta \ln p$	—	—	`pulsar_nav/ delay_estimation. py`

Chapter 7: Recursive Estimation (Extended Kalman Filter)

7.1 System Dynamics

Eq.	Description	Formula	Variables	Units	Implementation
7.1	Relative dynamics	$\Delta \dot{X} = A \cdot \Delta X + B \cdot \Delta a + G \cdot w_v$	$\Delta X = [\Delta x; \Delta v]$	m, m/s	`pulsar_nav/ kalman_filter.py`
7.2	System matrices	$A = \begin{bmatrix} 0 & I \\ 0 & 0 \end{bmatrix}, B = \begin{bmatrix} 0 \\ I \end{bmatrix}, G = \begin{bmatrix} I \\ 0 \end{bmatrix}$	$6 \times 6, 6 \times 3, 6 \times 3$	—	`pulsar_nav/ kalman_filter.py`
7.3	Process noise PSD (velocity)	$E[w_v(t)w_v(\tau)^T] = W_v \cdot \delta(t-\tau)$	W_v : velocity PSD	m ² /s	`pulsar_nav/ kalman_filter.py`
7.4	IMU measurement model	$a^{(i)}_{IMU} = a^{(i)} + w^{(i)}_a + b^{(i)}_a$	w_a : noise, b_a : bias	m/s ²	`pulsar_nav/ kalman_filter.py`
7.6	Relative IMU measurement	$\Delta a_{IMU} = \Delta a + b_a + w_a$	—	m/s ²	`pulsar_nav/ kalman_filter.py`
7.9	Bias dynamics (Brownian)	$\dot{b}_a = w_b$	w_b : bias drift noise	m/s ³	`pulsar_nav/ kalman_filter.py`
7.14	Error dynamics	$\dot{X}_e = A \cdot X_e + B \cdot b_a + B \cdot w_a + G \cdot w_v$	$X_e = \Delta X_{IMU} - \Delta X$	—	`pulsar_nav/ kalman_filter.py`

Eq.	Description	Formula	Variables	Units	Implementation
7.15	Clock differential dynamics	$\dot{t}_e = w_e$	w_e : clock noise	s/s	`pulsar_nav/ kalman_filter.py`
7.17	Full state vector (10-dim)	$X = [X_e; b_a; t_e]$	—	mixed	`pulsar_nav/ kalman_filter.py`
7.18	Continuous state dynamics	$\dot{X} = F \cdot X + w$	F : 10×10 system matrix	—	`pulsar_nav/ kalman_filter.py`
7.19	System matrix F	$F = \begin{bmatrix} A & B & 0 \\ 0 & 0 & 0 \end{bmatrix}$	10×10	—	`pulsar_nav/ kalman_filter.py`

7.2 Discrete-Time Dynamics

Eq.	Description	Formula	Variables	Units	Implementation
7.22	Discrete state equation	$X(k+1) = \Phi \cdot X(k) + w_d(k)$	Φ : state transition	—	`pulsar_nav/ kalman_filter.py`
7.23	State transition matrix	$\Phi = \exp(F \cdot T_s)$	T_s : sampling period	—	`pulsar_nav/ kalman_filter.py`
7.25	Φ expansion ($F^3=0$)	$\Phi = I + T_s \cdot F + (T_s^2/2) \cdot F^2$	—	—	`pulsar_nav/ kalman_filter.py`
7.26	Explicit Φ matrix	$\Phi = \begin{bmatrix} I & T_s \cdot I & \frac{1}{2} T_s^2 \cdot I & 0 \\ 0 & I & T_s \cdot I & 0 \\ 0 & 0 & I & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	10×10	—	`pulsar_nav/ kalman_filter.py`
7.29	Process noise covariance Q	See full matrix in book	10×10	mixed	`pulsar_nav/ kalman_filter.py`

7.3 Measurements

Eq.	Description	Formula	Variables	Units	Implementation
7.30	Pulse delay measurement	$\hat{c} \cdot \hat{t}_d^{(i)}(k) = H^{(i)} \cdot \Delta x(k) + c \cdot t_e(k) + \eta_d^{(i)}(k)$	$H^{(i)}$: pulsar direction	m	`pulsar_nav/ kalman_filter.py`
7.31	Measurement noise correlation	$E[\eta_d^{(i)}(k) \eta_d^{(j)}(k)] = R^{(i)} \cdot \delta_{ij}$	$R^{(i)} = c^2 \cdot \text{var}[\hat{t}_d^{(i)}]$	m ²	`pulsar_nav/ kalman_filter.py`
7.32	Measurement definition	$\hat{y}^{(i)}(k) \triangleq c \cdot \hat{t}_d^{(i)}(k)$	—	m	`pulsar_nav/ kalman_filter.py`
7.33	Discrete	$\hat{y}^{(i)}(k) =$	—	m	`pulsar_nav/`

Eq.	Description	Formula	Variables	Units	Implementation
	measurement model	$H^{(i)} \cdot \Delta x(k) + c \cdot t_e(k) + \eta_d^{(i)}(k)$			<code>kalman_filter.py</code>
7.34	Stacked measurements	$Y(k) = C \cdot \Delta X(k) + c \cdot 1 \cdot t_e(k) + \eta_d(k)$	$C: N \times 6$	m	<code>pulsar_nav/kalman_filter.py</code>
7.35	Measurement matrices	$C = [[H^{(1)}, 0], \dots, [H^{(N)}, 0]]$	$N \times 6$	—	<code>pulsar_nav/kalman_filter.py</code>
7.38	Measurement noise covariance	$R = \text{diag}(R^{(i)})$	$N \times N$	m^2	<code>pulsar_nav/kalman_filter.py</code>
7.39	Transformed measurement	$Z(k) = -(Y(k) - C \cdot \Delta X_{IMU}(k))$	—	m	<code>pulsar_nav/kalman_filter.py</code>
7.40	Final measurement equation	$Z(k) = H \cdot X(k) + \eta(k)$	—	m	<code>pulsar_nav/kalman_filter.py</code>
7.41	Full measurement matrix H	$H = [C, 0_{N \times 3}, -c \cdot 1]$	$N \times 10$	—	<code>pulsar_nav/kalman_filter.py</code>

7.4 Kalman Filter Equations

Eq.	Description	Formula	Variables	Units	Implementation
7.42a	Initial state estimate	$\hat{X}_0 = E[X_0]$	—	—	<code>pulsar_nav/kalman_filter.py</code>
7.42b	Initial error covariance	$P_0 = E[(X_0 - \hat{X}_0)(X_0 - \hat{X}_0)^T]$	—	—	<code>pulsar_nav/kalman_filter.py</code>
7.43a	Kalman gain	$K_k = P_k^- \cdot H^T \cdot [H \cdot P_k^- \cdot H^T + R]^{-1}$	—	—	<code>pulsar_nav/kalman_filter.py</code>
7.43b	State update	$\hat{X}_k = \hat{X}_k + K_k \cdot [Z_k - H \cdot \hat{X}_k]$	—	—	<code>pulsar_nav/kalman_filter.py</code>
7.43c	Covariance update (Joseph)	$P_k^+ = [I - K_k \cdot H] \cdot P_k^- \cdot [I - K_k \cdot H]^T + K_k \cdot R \cdot K_k^T$	—	—	<code>pulsar_nav/kalman_filter.py</code>
7.43d	State prediction	$\hat{X}_{k+1} = \Phi \cdot \hat{X}_k$	—	—	<code>pulsar_nav/kalman_filter.py</code>
7.43e	Covariance prediction	$P_{k+1} = \Phi \cdot P_k^+ \cdot \Phi^T + Q$	—	—	<code>pulsar_nav/kalman_filter.py</code>

7.5 Observability

Eq.	Description	Formula	Variables	Units	Implementation
7.45	Observability matrix	$O = [H; H \cdot F; H \cdot F^2]$	$3N \times 10$	—	<code>`pulsar_nav/ kalman_filter.py`</code>
7.46	Pulsar direction matrix	$\Gamma = [H^{\wedge}(1); H^{\wedge}(2); \dots; H^{\wedge}(N)]$	$N \times 3$	—	<code>`pulsar_nav/ kalman_filter.py`</code>

7.6 Absolute Navigation

Eq.	Description	Formula	Variables	Units	Implementation
—	SSB as reference	Place one spacecraft at SSB → absolute navigation	—	—	<code>`pulsar_nav/ kalman_filter.py`</code>

7.7 Geometric Velocity Estimation

Eq.	Description	Formula	Variables	Units	Implementation
7.48	Estimated time delay expansion	$\hat{t}_d^{(i)} \approx (\Delta \phi^{(i)} + n^{(i)}) / f_s^{(i)} \cdot (1 - V_j \cdot \cos \theta_i / c)$	—	s	<code>`pulsar_nav/ kalman_filter.py`</code>
7.52	Velocity estimation measurement	$\hat{z}^{(i)} = H^{(i)} \cdot \Delta x / c + t_e + z^{(i)} \cdot V_j \cdot \cos \theta_i / c + \eta^{(i)}$	—	s	<code>`pulsar_nav/ kalman_filter.py`</code>

7.8 Numerical Simulation Parameters

Parameter	Value	Description	Implementation
P_{0x}	$\text{diag}(10^6, 10^6, 10^6) \text{ m}$	Initial position uncertainty	<code>`pulsar_nav/kalman_filter.py`</code>
P_{0v}	$\text{diag}(10^4, 10^3, 10^2) \text{ m/s}$	Initial velocity uncertainty	<code>`pulsar_nav/kalman_filter.py`</code>
$P_{0\beta}$	$\text{diag}(10^{0.5}, 10^{0.5}, 10^{0.5}) \text{ m/s}^2$	Initial bias uncertainty	<code>`pulsar_nav/kalman_filter.py`</code>
P_{0t}	10^3 s	Initial clock uncertainty	<code>`pulsar_nav/kalman_filter.py`</code>
$\sqrt{W}_v$	$10^{-6} \cdot I \text{ m}/\sqrt{\text{s}}$	Velocity process noise	<code>`pulsar_nav/kalman_filter.py`</code>
$\sqrt{W}_a$	$10^{-7} \cdot I \text{ m}/\sqrt{\text{s}^3}$	Acceleration noise	<code>`pulsar_nav/kalman_filter.py`</code>
$\sqrt{W}_b$	$10^{-5} \cdot I \text{ m}/\sqrt{\text{s}^5}$	Bias drift noise	<code>`pulsar_nav/kalman_filter.py`</code>

Parameter	Value	Description	Implementation
$\sqrt{W}_e$	$10^{-6} \sqrt{s}$	Clock noise	`pulsar_nav/kalman_filter.py`

Pulsar Data (Tables 7.1, 7.2)

Pulsar	Period (s)	Gal. Long. (°)	Gal. Lat. (°)	Flux (ph/cm ² /s)
B0531+21	0.0335	184.56	-5.78	1.54E+00
B0540-69	0.0504	279.72	-31.52	5.15E-03
B0833-45	0.0893	263.55	-2.79	1.59E-03
B1509-58	0.1502	320.32	-1.16	1.62E-02
B1821-24	0.0031	7.80	-5.58	1.93E-04
B1937+21	0.0016	57.51	-0.29	4.99E-05
B1055-52	0.1971	164.50	-52.45	1.64E-06
J0437-47	0.0057	253.39	-41.96	6.65E-05

Legend

- ph/s = photons per second
- cycle = one full rotation of pulsar phase
- SSB = Solar System Barycenter
- TOA = Time of Arrival
- NHPP = Non-Homogeneous Poisson Process
- CRLB = Cramér–Rao Lower Bound
- CC = Cross Correlation
- NLS = Nonlinear Least Squares
- MLE = Maximum Likelihood Estimator
- EKF = Extended Kalman Filter
- IMU = Inertial Measurement Unit
- PSD = Power Spectral Density

Appendix B. Navigation Engine Reference

Navigation Engine

The core navigation engine lives in `pulsar_nav/navigation_engine.py`.

Model

For a spacecraft position vector $\mathbf{r}$ and a pulsar unit direction vector $\mathbf{n}$, the expected pulse timing delay is:

$$\text{delta_t} = (\mathbf{r} \cdot \mathbf{n}) / c$$

where $c = 299792.458 \text{ km/s}$.

The engine adds Gaussian timing noise in nanoseconds, then solves the inverse problem with linear least squares:

$$\mathbf{A} \mathbf{r} = \mathbf{c} * \text{delta_t}$$

$\mathbf{A}$ is the selected pulsar direction matrix. The solver requires at least four pulsars and supports 4-8 pulsars for the Monte Carlo study.

Regions

Synthetic spacecraft positions are generated in three regions:

- ``earth_orbit``: 6,700-42,200 km radius
- ``earth_moon``: 42,200-384,400 km radius
- ``deep_space``: 384,400-2,000,000 km radius

Default Simulation

```
python3 scripts/run_navigation_lab.py \  
--vectors output/ranked_pulsars.csv \  
--output-dir output \  
--trials 1000
```

Default timing noise levels:

10 ns, 50 ns, 100 ns, 500 ns, 1000 ns

Default pulsar counts:

4, 5, 6, 7, 8

Outputs

```
output/navigation_lab_results.csv  
output/navigation_lab_summary.csv  
output/navigation_lab_positions.csv
```

output/position_error_vs_timing_noise.svg

output/position_error_distribution.svg

output/position_error_vs_pulsar_count.svg

API

The web application exposes a simulation endpoint:

GET /api/navigation-lab?trials=1000&pulsars=6&noise=100®ion=earth_moon

The endpoint returns JSON suitable for the Navigation Lab dashboard page.

Appendix C. Source Directory Structure

PulsarNav AI

Autonomous Pulsar-Based Deep Space Navigation Framework

PulsarNav AI is a research-grade software framework for exploring autonomous spacecraft navigation using millisecond pulsars and real pulsar timing data. The project combines a Python scientific pipeline with a modern Next.js mission-control dashboard for catalog creation, TOA parsing, pulsar ranking, spacecraft simulation, least-squares navigation, error analysis, pulsar selection optimization, and 3D visualization.

The current implementation is designed for academic demonstrations, SAC/ISRO-style project presentations, and reproducible research prototypes using the NANOGrav 12.5-Year Narrowband Dataset.

Features

- Pulsar catalog builder from TEMPO/TEMPO2 `.par` files
- TOA database parser from NANOGrav narrowband `.tim` files
- RA/DEC and ecliptic-coordinate conversion to Cartesian unit vectors
- Pulsar ranking engine using timing precision, observation duration, TOA count, spin stability, and sky distribution
- Spacecraft position simulator for Earth orbit, Earth-Moon space, and deep-space regions
- Pulse propagation delay model using $\text{delay} = r \cdot n / c$
- Least-squares and weighted least-squares position estimation
- Monte Carlo error analysis across timing noise levels and pulsar counts
- Navigation Lab engine with 1000+ trial simulation runs, CSV exports, plots, and API access
- Pulsar selection optimization using geometry dilution of precision
- Research dashboard with dark aerospace mission-control UI
- 3D space visualization using Three.js and React Three Fiber
- Charts using Recharts and Plotly
- Lightweight sample CSV files for GitHub without committing the full dataset

Architecture

flowchart LR

A[NANOGrav Narrowband Dataset] --> B[PAR Parser]

A --> C[TIM Parser]

B --> D[Pulsar Catalog]

B --> E[Unit Vector Builder]

C --> F[TOA Database]

F --> G[TOA Statistics]

D --> H[Pulsar Ranking Engine]

E --> H

G --> H

H --> I[Pulsar Selection Optimization]

H --> J[Navigation Simulator]

J --> K[Least Squares Estimator]

K --> L[Error Analysis]

D --> M[Next.js Dashboard]

F --> M

H --> M

L --> M

I --> M

Folder Structure

```
.
├── app/                # Next.js App Router pages
│   ├── dashboard/      # Mission-control dashboard
│   ├── globals.css      # Tailwind/global styles
│   ├── layout.tsx
│   └── page.tsx         # Landing page
├── components/         # Reusable React UI, charts, 3D scene
├── docs/               # Project documentation and checklists
├── lib/               # Frontend utilities and demo mission data
├── pulsar_nav/         # Python navigation framework
│   ├── cli.py          # Pipeline CLI
│   ├── config.py
│   ├── coordinates.py
│   ├── parsing.py
│   ├── ranking.py
│   ├── simulation.py
│   └── visualization.py
├── sample_data/        # Small sample CSVs safe for GitHub
├── scripts/
│   └── run_pipeline.py  # Python pipeline launcher
└── .env.example
```

- |— .gitignore
- |— LICENSE
- |— package.json # Frontend dependencies and scripts
- |— requirements.txt # Python dependencies
- |— tailwind.config.ts
- |— tsconfig.json

Large local directories such as [NANOGrav_12yv4/](#), [dataset/](#), [output/](#), [.next/](#), and [node_modules/](#) are intentionally excluded from Git.

Appendix D. Python Module Inventory

This appendix lists the primary public functions and classes discovered in the `pulsar_nav` package. It is intended as a quick engineering reference for reviewers and future contributors.

Module	Classes	Functions
<code>__init__.py</code>	-	-
<code>cli.py</code>	-	<code>run_pipeline</code> , <code>write_report</code> , <code>build_parser</code> , <code>main</code>
<code>config.py</code>	-	<code>find_dataset_root</code> , <code>output_dir</code>
<code>coordinates.py</code>	<code>SkyCoord</code>	<code>parse_angle</code> , <code>ecliptic_to_equatorial</code> , <code>sky_to_unit_vector</code>
<code>crlb.py</code>	-	<code>fisher_integral_lp</code> , <code>fisher_matrix</code> , <code>crlb_phase</code> , <code>crlb_matrix</code> , <code>crlb_pulse_delay</code> , <code>crlb_distance_sigma</code> , <code>asymptotic_relative_efficiency</code> , <code>cc_nls_variance_phase</code> , <code>cc_nls_variance_delay</code>
<code>delay_estimation.py</code>	<code>PhaseEstimateResult</code> , <code>DelayEstimateResult</code> , <code>CrossCorrelationEstimator</code> , <code>NLSEstimator</code> , <code>MaximumLikelihoodEstimator</code>	<code>estimate_pulse_delay</code> , <code>run_delay_estimation</code> , <code>monte_carlo_phase_rms</code>
<code>epoch_folding.py</code>	-	<code>epoch_fold</code> , <code>epoch_folding_noise_variance</code> , <code>compute_empirical_noise_variance</code> , <code>frequency_shift_from_velocity_error</code> , <code>period_change_from_velocity_error</code> , <code>phase_change_from_velocity_error</code> , <code>velocity_error_tolerance</code> , <code>epoch_fold_with_velocity_error</code> , <code>noise_variance_with_velocity_error</code>
<code>kalman_filter.py</code>	<code>KalmanFilterConfig</code> , <code>FilterState</code> , <code>FilterHistory</code> , <code>PulsarNavigationKF</code>	<code>simulate_navigation</code> , <code>monte_carlo_navigation</code>
<code>navigation_engine.py</code>	<code>NavigationSimulationConfig</code>	<code>load_pulsar_vectors</code> , <code>generate_spacecraft_positions</code> , <code>expected_delays_s</code> , <code>add_timing_noise</code> , <code>estimate_position_least_squares</code> , <code>navigation_error_km</code> , <code>run_single_trial</code> , <code>run_monte_carlo</code> , <code>summarize_results</code> , <code>save_navigation_outputs</code> , <code>generate_navigation_plots</code> , <code>run_navigation_lab</code>
<code>navigation_geometry.py</code>	-	<code>galactic_to_cartesian</code> , <code>direction_matrix_Gamma</code> , <code>system_matrix_F</code> , <code>state_transition_Phi</code> , <code>measurement_matrix_H</code>

Module	Classes	Functions
		observability_matrix, check_observability, process_noise_Q, measurement_noise_R, geometric_dilution, position_dilution
parsing.py	-	pulsar_name_from_path, parse_par_file, build_catalog, build_unit_vectors, parse_tim_file, build_toa_database, toa_statistics
pulsar_catalog.py	PulsarEntry	get_pulsar, get_crab, direction_matrix, measurement_sigma_m
pulse_delay_estimation.py	PulseDelayEstimate	estimate_delay_cross_correlation, estimate_delay_nonlinear_least_squares, estimate_delay_maximum_likelihood
ranking.py	-	rank_pulsars, geometry_dop, select_best_geometry
signal_model.py	PulsarProfile, RateFunction	generate_photon_toas, make_rate_function, make_crab_rate_function
signal_processing.py	PulsarProfile	pulsar_rotation_phase, pulsar_intensity_at_time, generate_photon_toas_nhpp, epoch_fold_toas, epoch_fold_with_velocity_error, compute_time_to_arrival_model
simulation.py	-	random_positions, simulate_toa_delays, estimate_position, navigation_trial, monte_carlo
timing_model.py	-	earth_position_ssb_km, roemer_delay_ssb_s, dispersion_delay_s, compute_total_delay_s
visualization.py	-	create_static_dashboard
xray_nav.py	XRayPulsarProfile, CrossCorrelationEstimator, NonlinearLeastSquaresEstimator, MaximumLikelihoodEstimator, PulsarNavigationEKF	evaluate_gaussian_profile, get_default_profile, generate_photon_toas, epoch_folding, crlb_phase_error, crlb_pulse_delay_s